

Nahradte soubor title-pages.pdf
s Vašemi titulními stránkami
vygenerovanými ze STAGu.

Replace the title-pages.pdf
file with your own title pages
generated from STAG.

Digitální transformace procesů náboru a adaptace pracovníků ve zdravotnické organizaci

ABSTRAKT

V této diplomové práci analyzuji procesy náboru a adaptace pracovníků ve zdravotnické organizaci Krajská zdravotní, a.s., identifikuji jejich klíčová omezení a navrhuji digitální řešení pro jejich sjednocení a zefektivnění. Na základě analytických zjištění, rešerše existujících produktů a návrhu cílové architektury popisuji realizační i provozní aspekty systému včetně distribuované vrstvy inteligentního zpracování dat. Výsledkem je prakticky použitelný návrh, který respektuje multi-tenantní organizační model, požadavky na bezpečnost a auditovatelnost a on-premise provozní podmínky. Součástí práce je také vyhodnocení dalšího směřování vývoje s důrazem na stabilizaci provozu, měřitelnost a postupnou evoluci platformy.

Klíčová slova: Digitalizace, Optimalizace procesů, Softwarová architektura, Adaptační proces, Řízení lidských zdrojů

Digital transformation of recruitment and onboarding processes in a healthcare organization

ABSTRACT

In this diploma thesis, I analyze recruitment and employee onboarding processes in the Czech healthcare organization Krajská zdravotní, a.s., identify their key limitations, and propose a digital solution to unify and optimize them. Based on process analysis, market research of existing products, and target architecture design, I describe implementation and deployment aspects of the system, including a distributed intelligent data processing layer. The result is a practically applicable concept that respects a multi-tenant organizational model, security and auditability requirements, and on-premise operational constraints. The thesis also provides a roadmap for further development focused on operational stabilization, observability, and gradual platform evolution.

Keywords: Digitalization, Process Optimization, Software Architecture, Onboarding Process, Human Resource Management

PODĚKOVÁNÍ

Děkuji vedoucí své práce Ing. Jana Vitvarová Ph. D. za poskytnutou podporu a trpělivost. Mé díky také patří všem testerům za jejich zpětnou vazbu.

Bc. Pavel Vácha

OBSAH

Seznam tabulek	8
Seznam obrázků	10
Seznam zkratk	11
1 Úvod	12
2 Analýza současného stavu	13
2.1 Představení organizace	13
2.1.1 Organizační struktura z pohledu řízení lidských zdrojů	13
2.1.2 Specifika řízení lidských zdrojů ve zdravotnických organizacích	14
2.2 Současný stav procesů náboru pracovníků	15
2.2.1 Proces inzerce volných pozic	16
2.2.2 Proces příjmu a výběru potenciálních kandidátů	17
2.2.3 Proces náboru kandidáta	19
2.2.4 Proces zajištění vstupní agendy	20
2.2.5 Proces adaptace nových zaměstnanců	21
2.3 Identifikace problémů a úzkých míst	22
2.4 Požadavky na digitalizaci procesů	24
2.5 Specifikace funkcionálních a nefunkcionálních požadavků	26
2.5.1 Funkcionální požadavky	26
2.5.2 Nefunkcionální požadavky	29
3 Existující softwarová řešení	31
3.1 Metodika rešerše a hodnoticí rámec	31
3.2 Specializované ATS platformy	32
3.2.1 Teamio (Alma Career)	32
3.2.2 Recruitis	33
3.2.3 Datacruit	33
3.2.4 Sloneek (ATS modul)	33
3.3 Onboardingové platformy	34
3.3.1 Onbee	34
3.4 Integrované HCM suite (enterprise)	34
3.4.1 SAP SuccessFactors	34
3.4.2 Workday	35
3.4.3 Oracle Fusion Cloud HCM	35
3.5 Porovnání řešení vůči požadavkům KZ	35

3.6	Identifikovaná omezení dostupných produktů a volba cílového přístupu	37
4	Návrh softwarové architektury	39
4.1	Architektonické východisko a volba cílového stylu	39
4.2	Celkový obraz řešení	41
4.3	Struktura backendu	43
4.4	Datový model a integrační toky	45
4.5	Bezpečnost a provozní dohled	47
5	Implementace systému	49
5.1	Struktura řešení	49
5.2	Implementace backendu	50
5.2.1	Použité návrhové a integrační vzory v hiring_backend	51
5.2.2	Implementace doménové vrstvy	52
5.2.3	Implementace hexagonální architektury	53
5.2.4	Vynucení architektonických pravidel	53
5.2.5	Implementace spolehlivé asynchronní komunikace	54
5.2.6	Implementace auditní vrstvy	55
5.3	Implementace vrstvy inteligentního zpracování dat	55
5.4	Implementace datové vrstvy a migrací	57
5.4.1	Fyzický datový model	57
5.4.2	Migrace schématu	57
5.4.3	Perzistence dokumentů a infrastrukturní tabulky	58
5.4.4	Implementace bezpečnosti a identity	58
5.5	Implementace onboardingového portálu	59
5.6	Implementace kariérního portálu	59
5.6.1	Integrace analytického nástroje Umami	60
5.7	Implementace dohledové vrstvy	61
5.8	Komunikace s národními registry	61
6	Nasazení systému do cílového prostředí	63
6.1	Kontejnerizační model a síťová segmentace	63
6.2	Nasazení aplikačního stacku hiring_backend	65
6.3	Release workflow, distribuce image a aktualizace služeb	66
6.4	Správa konfigurace a bezpečnost provozu	67
6.5	Dohledová vrstva	67
6.6	Provozní omezení a mitigace	68
7	Uživatelské testování a zpětná vazba	69
7.1	Cíl a rámec pilotního ověření	69
7.2	Metodický postup a výběr respondentů	69
7.3	Testovací scénáře a hodnotící kritéria	70
7.4	Hlavní zjištění a implikace pro další rozvoj	72
8	Návrh dalšího směřování vývoje	74

8.1	Prioritizační rámec rozvoje	74
8.2	Krátkodobá fáze (0 - 6 měsíců)	74
8.3	Střednědobá fáze (6 - 18 měsíců)	75
8.4	Dlouhodobá fáze (18+ měsíců)	75
8.5	Řízení změny a organizační předpoklady	76
8.6	Závěrečné doporučení kapitoly	76
9	Závěr	77
	Použitá literatura	78

SEZNAM TABULEK

2.1 Klasifikace pozic v KZ	14
2.2 Proces P01 - Vystavení inzerátu	17
2.3 Proces P02 - Příjem a výber kandidátů k oslovení	18
2.4 Proces P03 - Pohovor a uzavření pracovního poměru	19
2.5 Proces P04 - Nástup zaměstnance	21
2.6 Proces P05 - Adaptace zaměstnance	22
2.7 Kvalitativní hodnocení závažnosti identifikovaných problémů	23
2.8 Požadavky na digitalizaci a jejich vztah na identifikované problémy	25
2.9 Funkcionální požadavky — Kariérní portál	26
2.10 Funkcionální požadavky — Interní řízení nábory	27
2.11 Funkcionální požadavky — Vstupní agenda a adaptace	28
2.12 Funkcionální požadavky — Integrace a ověřování kvalifikací	28
2.13 Funkcionální požadavky — Reporting a multi-tenantní správa	29
2.14 Nefunkcionální požadavky na systém	30
3.1 Hodnoticí rámec řešerše dostupných řešení	32
3.2 Porovnání vybraných řešení podle diskvalifikačních kritérií	36
3.3 Porovnání vybraných řešení podle doplňkových kritérií K6-K8	36
4.1 Porovnání architektonických variant	40

5.1 Implementační realizace hexagonální architektury backendu	51
5.2 Typy outbox událostí implementovaných v backendu	54
5.3 Globální role a jejich praktický význam v ReBAC modelu backendu	59
6.1 Hlavní služby v nasazení	64
6.2 Role komponent dohledové vrstvy	68
7.1 Zastoupení rolí v pilotním uživatelském ověření	70
7.2 Sada testovacích scénářů	71
7.3 Ukazatele použité v pilotním uživatelském ověření	72
7.4 Šablona souhrnných výsledků pilotního uživatelského ověření	73

SEZNAM OBRÁZKŮ

2.1 Diagram BPMN znázorňující proces	17
od žádosti po vystavení inzerátu	
2.2 Diagram BPMN znázorňující proces	18
od přijmutí přihlášky po pozvánku na pohovor	
2.3 Diagram BPMN znázorňující proces	20
od pohovoru po uzavření pracovního poměru	
2.4 Diagram BPMN znázorňující proces	21
od nového pracovního poměru po pří- pravu zaměstnance k výkonu práce	
2.5 Diagram BPMN znázorňující proces	22
od připraveného zaměstnance až po adaptovaného zaměstnance	
4.1 Systém v kontextu rolí a externích	41
služeb	
4.2 Logická kompozice řešení	42
4.3 Obecný princip hexagonální architek-	44
tury	
4.4 Návrh hranic backendu v hexagonál-	44
ním uspořádání	
4.5 Konceptuální ER diagram hlavních en-	46
tit a jejich vztahů	
4.6 Tok dat v monitorovací vrstvě od apli-	48
kace po vizualizaci	
5.1 Diagram implementace	50
5.2 Realizační tok Outbox-RabbitMQ a.vr-	56
stvy inteligentního zpracování dat v implementaci	
6.1 Obecný image-based release tok	66
služby s distribucí do registry a nasa- zením na cílový host	

SEZNAM ZKRATEK

KZ	Krajská zdravotní, a.s.
HR	Human Resources
LZ	Lékaři a farmaceuti
NL- ZP	Nelékařští zdravotničtí pracovníci
IT	Informační technologie
ÚZIS	Ústav zdravotnických informací a statistiky ČR
BO- ZP	Bezpečnost a ochrana zdraví při práci
THP	technickohospodářských pracovníků
ATS	Applicant Tracking System
HCM	Human Capital Management
SSO	Single Sign-On

1 ÚVOD

Digitalizace personálních procesů ve zdravotnictví není pouhým převodem formulářů do elektronické podoby. V prostředí velké zdravotnické organizace se v jednom procesu střetává vysoký objem náboru, legislativní požadavky na odbornou způsobilost, rozdílné potřeby jednotlivých pracovišť i tlak na rychlé obsazování pozic. Pokud jsou tyto okolnosti dlouhodobě řešeny převážně e-mailem, sdílenými tabulkami a lokálními zvyklostmi, nevzniká jen administrativní zátěž. Vzniká také nepřehlednost, která zpomaluje nábor, komplikuje nástup nových zaměstnanců a oslabuje schopnost řídit proces na úrovni celé organizace.

Téma této práce proto vychází z praktické potřeby Krajské zdravotní, a.s. sjednotit a zpřehlednit nábor a adaptaci pracovníků napříč odštěpnými závody. Nejde přitom jen o zefektivnění jednotlivých dílčích kroků, ale o vytvoření takového řešení, které propojí inzerci, práci s uchazeči, vstupní agendu i adaptaci nového zaměstnance do jednoho srozumitelného a provozně udržitelného celku.

Cílem práce je analyzovat současný stav těchto procesů, identifikovat jejich hlavní nedostatky a navrhnout řešení jejich digitalizace formou odpovídající softwarové architektury. Součástí práce je i porovnání existujících produktů a zdůvodnění, proč v podmínkách Krajské zdravotní, a.s. dává smysl vlastní vývoj. Na analytickou a řešeršní část navazuje návrh architektury, popis implementace, nasazení do cílového prostředí a pilotní uživatelské ověření řešení. Práce se soustředí na proces od vzniku personální potřeby až po adaptaci nového zaměstnance.

2 ANALÝZA SOUČASNÉHO STAVU

Efektivní digitalizaci nelze stavět na domněnkách o tom, jak organizace funguje, ale na přesném porozumění skutečnému průběhu procesů, jejich aktérům a slabým místům. V souladu s metodikou procesního řízení dle Řepy [1] proto analytická část nejprve mapuje výchozí stav a teprve na tomto základě formuluje požadavky na budoucí informační systém.

Analýza současného stavu procesů náboru a adaptace pracovníků v Krajské zdravotní, a.s. proto nejprve ukotvuje organizační a doménová specifika zdravotnického prostředí. Následně zachycuje klíčové procesy pomocí notace BPMN (Business Process Model and Notation), identifikuje jejich úzká místa a převádí zjištěné nedostatky do podoby funkčních a nefunkčních požadavků.

2.1 PŘEDSTAVENÍ ORGANIZACE

Krajská zdravotní, a.s. (KZ) je akciová společnost vlastněná Ústeckým krajem a současně největší poskytovatel lůžkové i ambulantní zdravotní péče v regionu. Vznikla v roce 2007 sloučením pěti nemocnic do jediné právní entity. Smyslem tohoto kroku nebyla pouze organizační konsolidace, ale i sjednocení řízení, nákupu a sdílení odborných kapacit napříč regionem.

V roce 2021 se struktura KZ dále rozšířila. Do společnosti byla integrována **Nemocnice Litoměřice, o.z.**, a následně bylo převzato i zdravotnické zařízení v Rumburku, které dnes funguje jako **KZ - Masarykova nemocnice v Ústí nad Labem, o.z., detašované pracoviště Rumburk**. Tím se počet odštěpných závodů zvýšil na sedm.

S více než jedenácti tisíci zaměstnanci patří KZ mezi největší zaměstnavatele v Ústeckém kraji. Pro tuto práci je podstatné zejména to, že značnou část personálu tvoří zdravotničtí pracovníci s regulovanou odbornou způsobilostí. Nábor a adaptace zde proto nejsou jen administrativní agendou, ale procesem, který musí současně splnit provozní, legislativní i odborné požadavky.

2.1.1 ORGANIZAČNÍ STRUKTURA Z POHLEDU ŘÍZENÍ LIDSKÝCH ZDROJŮ

Organizační uspořádání KZ kombinuje centrální řízení s vysokou mírou provozní autonomie jednotlivých nemocnic. Každý odštěpný závod má vlastní personální zázemí Human Resources (HR) pro operativní agendu, správu smluv, docházku

i řízení adaptace, ale současně se řídí jednotnou metodikou a strategickými cíli holdingu.

Z pohledu digitalizace je důležité, že systém nesmí uvažovat organizaci jako jeden homogenní celek. Musí respektovat samostatnost jednotlivých závodů a zároveň umožnit centrální dohled, reporting a správu pravidel. V architektonické terminologii jde o požadavek na *multi-tenantní* řešení, ve kterém každý odštěpný závod vystupuje jako samostatný tenant sdílející společnou infrastrukturu i metodické řízení.

2.1.2 SPECIFIKA ŘÍZENÍ LIDSKÝCH ZDROJŮ VE ZDRAVOTNICKÝCH ORGANIZACÍCH

Řízení lidských zdrojů v prostředí KZ se od běžného korporátního modelu liší především tím, že velká část pracovních rolí podléhá regulované odborné způsobilosti. Náborový systém proto nemůže řešit pouze evidenci kandidátů a komunikaci s nimi. Musí současně hlídat kvalifikace, zákonné podmínky výkonu povolání a návaznost těchto informací na adaptační proces.

Rozdílné nároky se projevují už v samotné kategorizaci pracovníků. Zaměstnanci KZ spadají do několika skupin, z nichž každá vyžaduje jiný rozsah dokladů, jiný způsob ověření způsobilosti a často i odlišný průběh adaptace. Pro další text zavádím zejména kategorie Lékaři a farmaceuti (LZ), Nelékařští zdravotničtí pracovníci (NLZP) a Informační technologie (IT), které se v procesech opakují.

Tabulka 2. 1: Klasifikace pozic v KZ

Kategorie (dle KS)	Typické pozice v KZ	Klíčová specifika pro HR
LZ	Atestovaní lékaři, lékaři v přípravě, farmaceuti	Sledování specializačního vzdělávání (atestace), IP-VZ, evidence v ČLK.
NLZP	Všeobecné sestry, dětské sestry, porodní asistentky	Odborná a specializovaná způsobilost, vzdělávání pod NCONZO.
Ostatní NLZP a JOP	Radiologičtí asistenti, fyzioterapeuti, sanitáři, kliničtí psychologové	Certifikované kurzy, akreditované stáže, specifické nároky na praxi.
THP a Provoz	Ekonomové, IT specialisté, údržba, stravovací provoz	Zařazení dle Katalogu prací, standardní zákoník práce.

Hlavním specifikem je právě práce v rámci **regulované odborné způsobilosti**. Základním pilířem HR procesů v KZ je soulad s legislativním rámcem pro výkon

zdravotnických povolání. Nábor a následná správa zaměstnanců se zde opírají zejména o dvě normy:

- Zákon č. 95/2004 Sb. - upravuje získávání odborné a specializované způsobilosti u lékařů, zubních lékařů a farmaceutů. Systém musí sledovat průběh předatestační přípravy, platnost členství v ČLK/ČSK a zařazení do specializačních oborů.
- Zákon č. 96/2004 Sb. - definuje podmínky pro NLZP. Zde je kritické sledování odborné způsobilosti a schopnosti vykonávat povolání bez odborného dohledu (v souladu s aktuální metodikou Ministerstva zdravotnictví a Ústav zdravotnických informací a statistiky ČR (ÚZIS)).

Informační systém proto musí umět validovat doklady o dosaženém vzdělání, zaznamenat výsledek jejich kontroly a následně pracovat i s informacemi o registracích v profesních komorách nebo o odborné způsobilosti pod dohledem. Bez této vrstvy by digitalizace sice urychlila administrativu, ale neřešila by to, co je v prostředí zdravotnické organizace skutečně kritické.

Dalším specifikem je **kontinuální nábor**. Na rozdíl od prostředí, kde se nábor spouští jen při výjimečné potřebě, musí zdravotnická organizace dlouhodobě reagovat na fluktuaci, demografický vývoj i celorepublikový nedostatek pracovníků, zejména mimo hlavní centra. [2] Systém proto musí být připraven na opakované a souběžné zpracování velkého množství pozic i uchazečů.

Po náboru navíc nekončí práce organizace podpisem pracovní smlouvy. Následuje **vícetupňový adaptační proces**, který zahrnuje nejen standardní nástupní agendu a školení Bezpečnost a ochrana zdraví při práci (BOZP), ale i odborné zapracování, práci s interní dokumentací, seznámení s nemocničními systémy a ověření připravenosti na konkrétním pracovišti. Právě vazba mezi nábořem, vstupní agendou a adaptací je proto v této práci klíčová.

2.2 SOUČASNÝ STAV PROCESŮ NÁBORU PRACOVNÍKŮ

Před zahájením digitalizace stojí správa uchazečů v KZ převážně na „svaté dvojici“ sdílených tabulek a e-mailové komunikace, kterou doplňují lokální zvyklosti jednotlivých závodů. Takové řešení může fungovat v menší organizaci, ale v podmínkách zdravotnického holdingu začíná postupně brzdit celý proces. Nábor zde pak nepůsobí jako jeden plynulý celek, ale jako řada navazujících administrativních kroků, mezi nimiž se informace ztrácejí nebo zbytečně přepisují.

Pro účely analýzy jsem proto mapoval skutečný průběh procesů pomocí strukturovaných rozhovorů s vedoucím personálního oddělení a s náboráři jednotlivých odštěpných závodů. Vycházel jsem nejen z formální dokumentace, ale i z popisu

reálné praxe, tedy z toho, jak proces skutečně probíhá při běžném provozu, včetně neformálních dohod a obcházení chybějících nástrojů.

Pro zajištění terminologické konzistence jsou v této kapitole používány následující pojmy:

- **Uchazeč** o zaměstnání je fyzická osoba, která reaguje na konkrétní zveřejněnou pracovní pozici a doručí zaměstnavateli svou přihlášku (životopis, motivační dopis nebo jinou formu reakce).
- **Kandidát** je uchazeč, který prošel prvotním screeningem a byl vybrán do další fáze výběrového řízení (např. k osobnímu pohovoru nebo odbornému posouzení).
- **Zájemce** o zaměstnání je osoba, která projeví zájem o zaměstnání v KZ bez vazby na konkrétní vyhlášenou pracovní pozici (tzv. talent pool).

2.2.1 PROCES INZERCE VOLNÝCH POZIC

Inzerce volných pracovních pozic je v současném stavu rozložena mezi několik nezávislých kanálů:

- **Webové portály třetích stran** - Jobs.cz, Práce.cz (provozovatel LMC), portál MPSV
- **Nemocniční nástěnky** - fyzické nástěnky v areálech nemocnic
- **Webové stránky nemocnic** - statické stránky s omezenou aktualizací
- **Sociální síť** — LinkedIn

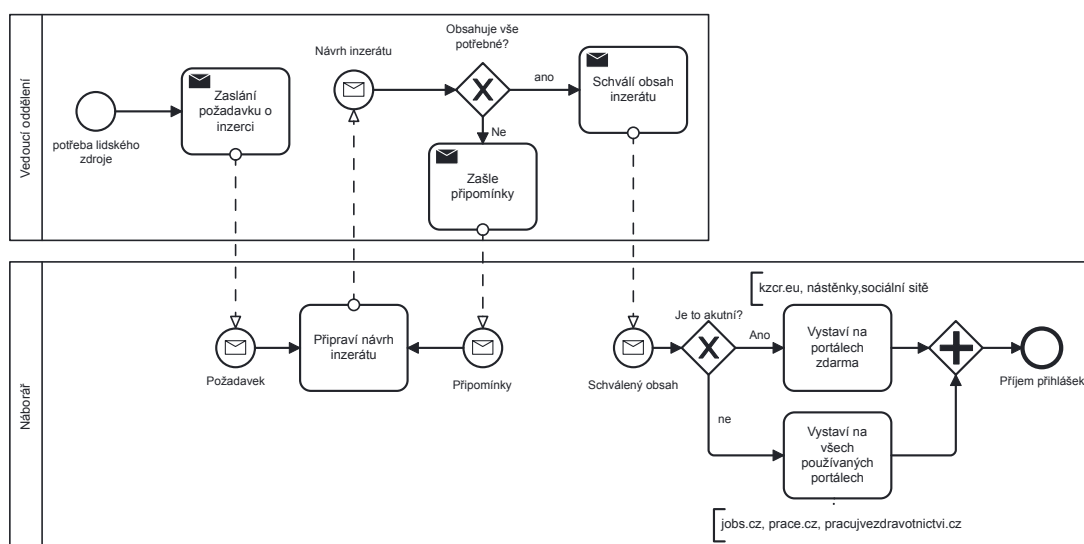
Absence vlastního kariérního portálu znamená, že uchazeč nemá jednotný vstupní bod, ve kterém by našel aktuální nabídky všech závodů, základní informace o zaměstnavateli i možnost okamžitě reagovat na pozici. Už na této úrovni tedy vzniká roztříštěnost, kterou organizace dále přenáší i do interní práce s inzeráty.

Proces začíná tím, že vedoucí oddělení identifikuje potřebu obsadit pozici a předává ji personálnímu oddělení. Vstupní informace však nejsou standardizované. Rozsah i kvalita zadání se liší podle konkrétního vedoucího, takže HR často nejprve doplňuje chybějící údaje a až poté připravuje text inzerátu. Samotné schvalování proto probíhá v několika e-mailových iteracích a místo plynulého workflow vzniká komunikační smyčka.

Po schválení textu následuje ruční publikace na jednotlivých platformách. Každý portál má vlastní formulář, vlastní strukturu dat i vlastní způsob následné aktualizace. Změna termínu nebo úprava požadavků proto neznamena jednu opravu v centrálním systému, ale opakování stejné práce na více místech. Důsledkem je nízká transparentnost, vyšší časová náročnost a slabý přehled vedoucích pracovníků o tom, kde se jejich pozice právě nachází a jaké reakce na ni přišly.

Tabulka 2. 2: Proces P01 - Vystavení inzerátu

Identifikátor procesu:	P01
Název procesu:	Vytvoření inzerátu pro pracovní pozici
Zákazník:	Vedoucí daného oddělení v odštěpném závodě
Vlastník procesu:	Náborář, vedoucí daného oddělení v odštěpném závodě
Účel:	Vytvoření inzerátu na poptávanou pozici
Produkt:	Inzerát
Technické prostředky:	E-mail, webové stránky KZ, webové portály třetích stran, nemocniční nástěnky, sociální sítě
Metrika:	Rychlost od žádosti po vystavení inzerátu
Nedostatky:	Manuální publikace inzerátu, již neaktuální inzeráty na portálech, nejednotý přehled o vydaných financích, dlouhé administrativní kolečko mezi vedoucím a náborářem



Obrázek 2. 1: Diagram BPMN znázorňující proces od žádosti po vystavení inzerátu

2.2.2 PROCES PŘÍJMU A VÝBĚRU POTENCIÁLNÍCH KANDIDÁTŮ

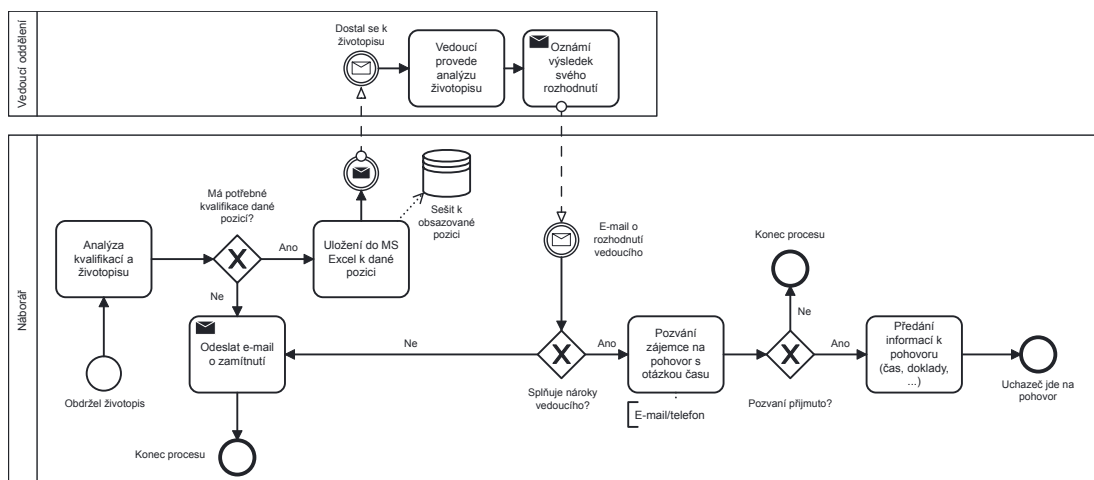
Přihlášky dnes přicházejí z různých zdrojů a náborář je musí ručně sjednocovat do tabulek a lokálně uložených příloh. Samotné získání přehledu o kandidátech je tak oddělenou administrativní činností, nikoli přirozenou součástí náborového systému. Výsledkem je nekonzistence dat mezi pracovišti a nízká dohledatelnost dokumentů.

Prvotní screening uchazečů probíhá převážně manuálně. U zdravotnických pozic to znamená vyhledávat v každém životopise údaje o vzdělání, odborné způsobilosti a další podmínky, které rozhodují o dalším postupu. Bez automatických filtrů a strukturovaných dat je tento krok pomalý a zároveň náchylný k přehlédnutí důležitých informací. Užší výběr se následně předává vedoucím oddělení znovu e-mailem, takže rozhodovací proces zůstává roztříštěný.

Kritickým nedostatkem je i absence systematické databáze uchazečů a zájemců. Informace o kandidátech, kteří nebyli přijati nebo reagovali bez vazby na konkrétní pozici, se dále nevyužívají. Organizace tím přichází o možnost vracet se k relevantním kontaktům při dalším náboru a zvyšuje si závislost na opakované inzerci.

Tabulka 2. 3: Proces P02 - Příjem a výběr kandidátů k oslovení

Identifikátor procesu:	P02
Název procesu:	Příjem přihlášek a výběr kandidátů k oslovení
Zákazník:	Vedoucí daného oddělení v odštěpném závodě
Vlastník procesu:	Náborář, vedoucí daného oddělení v odštěpném závodě
Účel:	Výběr kandidátů, jenž splňují požadavky obsazované pozice
Produkt:	Kandidát pozvaný na pohovor
Technické prostředky:	E-mail, telefon, tabulkový procesor, Webex
Metrika:	Rychlost od vystavení inzerátu po oslovení kandidáta
Nedostatky:	Neexistence centrální databáze uchazečů o pozici, neexistence databáze potencionálních zájemců o zaměstnání v KZ



Obrázek 2. 2: Diagram BPMN znázorňující proces od přijetí přihlášky po pozvání na pohovor

2.2.3 PROCES NÁBORU KANDIDÁTA

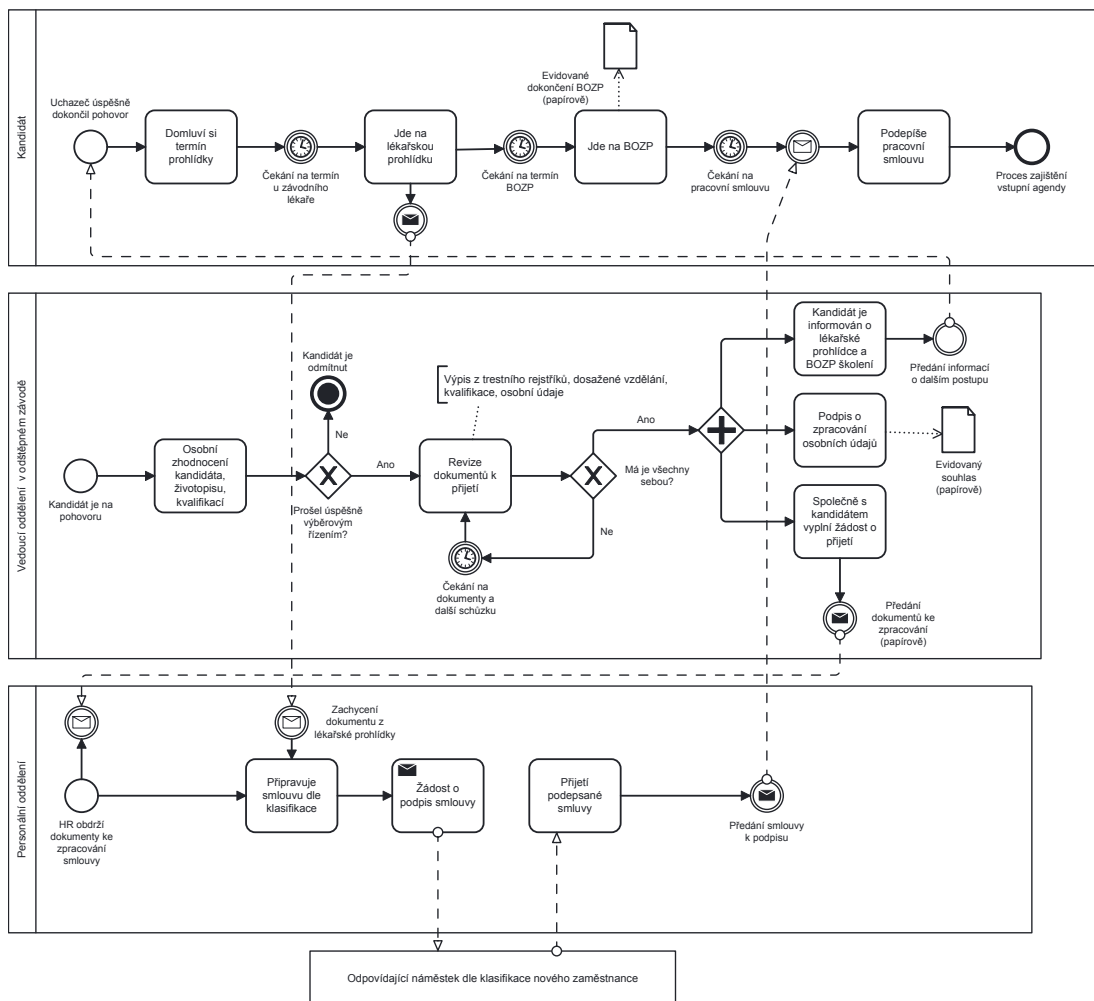
Proces náboru pracovníka (P03) navazuje na okamžik, kdy je kandidát pozván na pohovor. V této fázi už nejde pouze o výběr vhodného člověka, ale o splnění všech podmínek nutných pro vznik pracovněprávního vztahu. Prakticky se zde propojuje rozhodnutí vedoucího oddělení s administrativními a zákonnými kroky personálního oddělení.

Po úspěšném pohovoru začíná sběr identifikačních údajů a povinných dokumentů. Kandidát dokládá výpis z rejstříku trestů, doklady o vzdělání, odborné kvalifikaci a další podklady podle charakteru pozice. Následují vstupní lékařská prohlídka a školení BOZP. Každý z těchto kroků je nezbytný, ale v současném stavu probíhá převážně manuálně a bez jednotného přehledu o tom, co již bylo dodáno a co ještě chybí.

Proces tak sice formálně vede k uzavření pracovní smlouvy, ale z provozního hlediska v něm chybí centrální koordinace. Informace jsou rozloženy mezi e-maily, listinné podklady a jednotlivé pracovníky, což prodlužuje průchod celou fází a zvyšuje riziko administrativního zdržení.

Tabulka 2. 4: Proces P03 - Pohovor a uzavření pracovního poměru

Identifikátor procesu:	P03
Název procesu:	Pohovor a uzavření pracovního poměru
Zákazník:	Vedoucí oddělení, PAM
Vlastník procesu:	Vedoucí oddělení, personální a mzdové oddělení
Účel:	Odborné posouzení kandidáta a splnění podmínek pro vznik pracovněprávního vztahu
Produkt:	Podepsaná pracovní smlouva
Technické prostředky:	Listinné dokumenty, e-mail, telefon
Metrika:	Doba od pohovoru po podpis pracovní smlouvy
Nedostatky:	Manuální sběr dokladů, vícestupňová administrativa, papírová komunikace



Obrázek 2. 3: Diagram BPMN znázorňující proces od pohovoru po uzavření pracovního poměru

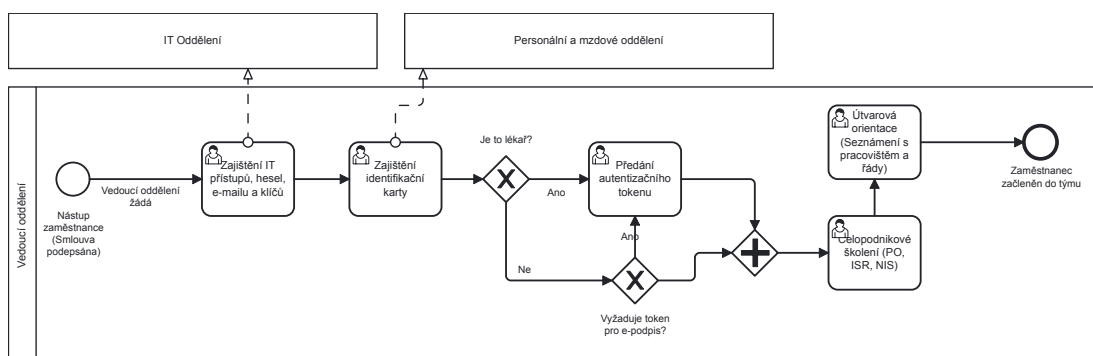
2.2.4 PROCES ZAJIŠTĚNÍ VSTUPNÍ AGENDY

V okamžiku nástupu zaměstnance začíná fáze, ve které je třeba připravit člověka na skutečný výkon práce. Patří sem vydání identifikační karty, nastavení docházky, přístupových oprávnění a v některých případech i prostředků pro elektronické podepisování. Z pohledu procesu je důležité, že tyto kroky nevykonává jediná role, ale více útvarů s rozdílnou odpovědností.

Vedle technických přístupů probíhá i útvarová orientace nového zaměstnance. Vedoucí pracovník jej seznamuje s pracovištěm, provozním řádem, dokumentací a očekáváním spojeným s výkonem práce. Problém současného stavu nespočívá v tom, že by tyto kroky neexistovaly, ale v tom, že nejsou vedeny v jednotném přehledu. Organizace proto obtížně zjišťuje, co bylo splněno, co se zdrželo a kde se nový zaměstnanec může zaseknout ještě před plným nástupem do role.

Tabulka 2. 5: Proces P04 - Nástup zaměstnance

Identifikátor procesu:	P04
Název procesu:	Nástup zaměstnance a zajištění přístupových prostředků
Zákazník:	Nový zaměstnanec
Vlastník procesu:	PAM, IT, vedoucí oddělení
Účel:	Zajištění připravenosti zaměstnance k výkonu práce
Produkt:	Zaměstnanec vybaven ID kartou, přístupy a případně tokenem
Technické prostředky:	ID karta, docházkový systém, autentizační token, interní IT systémy
Metrika:	Doba od podpisu smlouvy po plnou funkčnost zaměstnance
Nedostatky:	Oddělené nastavování oprávnění, ruční aktivace přístupů, absence jednotného postupu



Obrázek 2. 4: Diagram BPMN znázorňující proces od nového pracovního poměru po přípravu zaměstnance k výkonu práce

2.2.5 PROCES ADAPTACE NOVÝCH ZAMĚSTNANCŮ

Adaptační proces tvoří most mezi formálním nástupem a plnohodnotným výkonem práce. V prostředí KZ je tato fáze zvláště důležitá u zdravotnických pozic, protože nestačí pouze administrativně dokončit nástup. Je třeba zajistit i odborné zapracování v podmínkách konkrétního pracoviště.

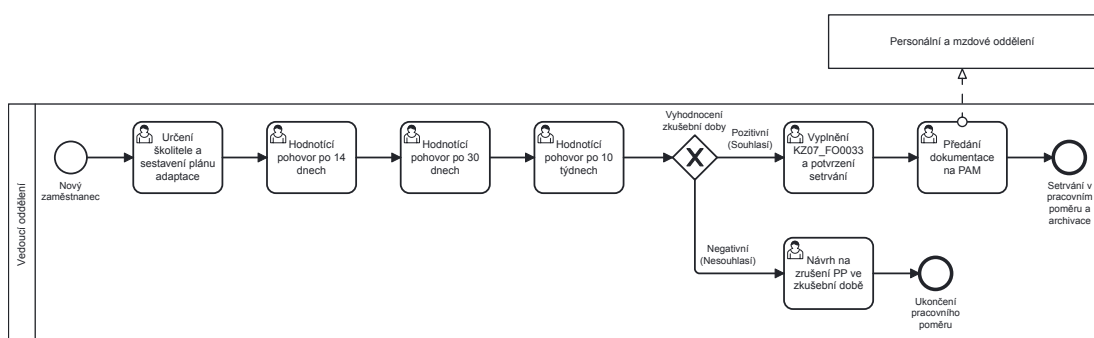
Obecná část adaptace se vztahuje na všechny zaměstnance a probíhá zejména ve zkušební době. Vedoucí zaměstnanec určuje školitele, nastavuje adaptační plán a průběžně hodnotí jeho plnění. Odborná část je pak výraznější u zdravotnických profesí, kde se sleduje nejen zvládnutí organizačních pravidel, ale i osvojení pracovních postupů, standardů a očekávané míry samostatnosti.

V současném stavu je však adaptace z velké části vedena papírově. To komplikuje průběžný monitoring, znesnadňuje auditní dohled a prakticky znemožňuje

centrálně vyhodnocovat, jak adaptace probíhá napříč odštěpnými závody. Organizace tak sice adaptaci vykonává, ale nemá k dispozici jednotný digitální obraz o jejím stavu a výsledcích.

Tabulka 2. 6: Proces P05 - Adaptace zaměstnance

Identifikátor procesu:	P05
Název procesu:	Adaptační proces zaměstnance
Zákazník:	Vedoucí oddělení, zaměstnanec
Vlastník procesu:	Vedoucí zaměstnanec, školitel
Účel:	Začlenění zaměstnance do pracovního a odborného prostředí
Produkt:	Plně adaptovaný zaměstnanec
Technické prostředky:	Adaptační formuláře, hodnotící pohovory
Metrika:	Míra setrvání po zkušební době, fluktuace
Nedostatky:	Papírová dokumentace, chybějící monitoring, slabý reporting



Obrázek 2. 5: Diagram BPMN znázorňující proces od připraveného zaměstnance až po adaptovaného zaměstnance

2.3 IDENTIFIKACE PROBLÉMŮ A ÚZKÝCH MÍST

Na základě analýzy současného stavu procesů nábory a adaptace v KZ, konzultací s HR pracovníky jednotlivých nemocnic a pozorování reálného průběhu procesů jsem identifikoval následující klíčové problémy. Problémy jsou kategorizovány podle oblastí dopadu a doplněny o kvalitativní hodnocení závažnosti.

Identifikované problémy mají kumulativní charakter a v provozní praxi se vzájemně zesilují. Významným akceleračním faktorem je vysoká personální dynamika organizace, která dosahuje průměrně 110 nástupů měsíčně (v sezónních špičkách až 180). Typická měsíční struktura nástupů přitom zahrnuje přibližně 10 pracovníků kategorie LZ, 50 NLZP a 50 technickohospodářských pracovníků

(THP)/dělnických profesí. V takovém objemu se manuální administrativa stává kritickým omezením propustnosti celého procesu.

Z analytického pohledu lze problémy rozdělit do tří tematických oblastí. První oblast představuje **datová fragmentace** (P1-P3), kdy jsou informace rozptýleny mezi e-mailovou komunikací, lokální soubory a tabulkové přehledy jednotlivých závodů. Druhou oblast tvoří **procesní a řídicí nedostatky** (P4-P9), zejména chybějící auditní stopa, nízká standardizace rozhodování a slabá transparentnost stavu náboru i vstupní agendy. Třetí oblast je **digitální nepodpořená adaptace** (P10), která omezuje možnost systematicky řídit zapracování nových zaměstnanců a vyhodnocovat jeho úspěšnost.

Tabulka 2. 7: Kvalitativní hodnocení závažnosti identifikovaných problémů

ID	Problém	Závažnost	Hlavní dopad
P1	Tabulková a papírová agenda	Vysoká	Riziko ztráty dat, neefektivní práce
P2	Chybějící evidence uchazečů	Vysoká	Ztráta kandidátů, nemožnost analytiky
P3	Neexistující evidence zájemců	Střední	Ztráta potenciálních kandidátů
P4	Absence auditní stopy	Vysoká	Právní rizika, GDPR, neschopnost auditu
P5	Omezený reporting	Střední	Rozhodování bez datové opory
P6	Komunikační smyčka	Vysoká	Prodloužení doby obsazení pozice
P7	Manuální publikace	Střední	Časová náročnost, nekonzistence
P8	Nestrukturované hodnocení	Střední	Subjektivní výběr
P9	Nepřehledný stav vstupní agendy	Vysoká	Problémy při nástupu zaměstnance
P10	Papírová adaptace	Vysoká	Nemožnost monitoringu a reportingu

Souhrnně lze konstatovat, že stávající procesní model již neodpovídá rozsahu organizace ani požadavkům na datově řízené rozhodování. Zjištění této kapitoly proto představují přímý vstup pro definici požadavků na cílové řešení.

2.4 POŽADAVKY NA DIGITALIZACI PROCESŮ

Na základě provedené analýzy procesů a identifikovaných problémů (Kapitola 2.3) lze formulovat klíčové požadavky na digitalizaci procesů náboru a adaptace. Každý požadavek je odůvodněn vazbou na identifikované problémy.

Požadavky R1-R6 představují minimální funkční rámec cílového systému. Každý požadavek je formulován tak, aby byl implementovatelný, ověřitelný při testování a současně jednoznačně navázaný na problémy identifikované v předchozí sekci.

Tabulka 2. 8: Požadavky na digitalizaci a jejich vazba na identifikované problémy

ID	Požadavek	Vymezení a cíl	Vazba
R1	Multi-tenantní architektura	Respektovat holdingovou strukturu KZ (závody jako samostatní tenanti) a současně umožnit centrální řízení a reporting.	P1, P2, P5
R2	Veřejný kariérní portál	Poskytnout jednotný vstupní bod s aktuálními nabídkami, online přihláškou a registrací zájemce (talent pool).	P2, P3, P6, P7
R3	Interní administrační rozhraní	Centralizovat správu inzerce, kandidátů, výběrových kroků, kontrolu kvalifikací i přípravu vstupní agendy.	P1, P2, P4, P5, P6, P7, P8, P9
R4	Adaptační portál	Digitalizovat adaptační plány, průběžné hodnocení, notifikace termínů a souhrnný přehled stavu adaptace.	P1, P4, P5, P10
R5	Integrace s NRZP	Automatizovat ověřování odborné způsobilosti zdravotnických pracovníků přes napojení na NRZP.	P1, P4
R6	Reporting a analytika	Zajistit dashboardy a export metrik náboru a adaptace pro závody i centrální úroveň.	P5

2.5 SPECIFIKACE FUNKCIONÁLNÍCH A NEFUNKCIONÁLNÍCH POŽADAVKŮ

Na základě formulovaných požadavků na digitalizaci (R1-R6) jsem v této sekci provedl jejich dekompozici na konkrétní funkcionální a nefunkcionální požadavky, které slouží jako vstup pro návrh softwarové architektury a implementaci systému.

2.5.1 FUNKCIONÁLNÍ POŽADAVKY

Funkcionální požadavky definují konkrétní chování systému, tedy co systém musí umožňovat svým uživatelům nebo jakých výstupů musí být schopen. Požadavky jsou kategorizovány podle oblastí systému a prioritizovány metodou MoSCoW (Must have, Should have, Could have, Won't have).

Tabulka 2. 9: Funkcionální požadavky — Kariérní portál

ID	Požadavek	Priorita	Vazba
F01	Systém zobrazí veřejný seznam aktuálních pracovních nabídek ze všech odštěpných závodů KZ s možností filtrování podle závodu, kategorie pozice, typu úvazku a lokality	Must	R2
F02	Uchazeč se může přihlásit na vybranou pozici prostřednictvím online formuláře s přiložením životopisu a dalších dokumentů	Must	R2
F03	Systém umožní registraci zájemce o zaměstnání v KZ i bez vazby na konkrétní pozici (talent pool)	Must	R2, R3
F04	Kariérní portál prezentuje informace o zaměstnavatelských benefitech, stipendijních programech a pracovním prostředí v KZ	Should	R2
F05	Detail pracovní nabídky obsahuje strukturovaný popis pozice, požadavky na kvalifikaci, nabízené podmínky a kontaktní informace	Must	R2

Tabulka 2. 10: Funkcionální požadavky — Interní řízení náboru

ID	Požadavek	Priorita	Vazba
F06	Systém umožní založení a správu požadavku na obsazení pozice včetně schvalovacího workflow mezi vedoucím oddělení a HR	Must	R3
F07	Systém bude vést centrální evidenci kandidátů s historií změn stavů napříč celým náborovým procesem	Must	R3
F08	Systém umožní plánování pohovorů, přiřazení odpovědných osob a evidenci výsledků jednotlivých kol	Must	R3
F09	Systém poskytne standardizované komunikační šablony (pozvánka, zamítnutí, žádost o doplnění podkladů) a jejich evidenci	Should	R3
F10	Systém umožní strukturované hodnocení kandidátů dle předem definovaných kritérií	Should	R3, R6
F11	Vedoucí oddělení bude mít online přehled o stavu svých náborových požadavků bez nutnosti ad-hoc e-mailových dotazů	Must	R3, R6

Tabulka 2. 11: Funkcionální požadavky — Vstupní agenda a adaptace

ID	Požadavek	Priorita	Vazba
F13	Systém vytvoří vstupní checklist nástupu podle typu pozice a organizační jednotky	Must	R3, R4
F14	Systém umožní evidovat plnění přednástupních povinností (BOZP, vstupní prohlídka, smluvní dokumentace, identifikační karta, přístupová oprávnění)	Must	R3, R4
F15	Systém bude automaticky upozorňovat odpovědné role na blížící se termíny nebo prodlení u vstupní agendy	Should	R4
F16	Vedoucí nebo školitel bude moci založit adaptační plán obsahující úkoly, termíny a hodnoticí milníky	Must	R4
F17	Systém umožní průběžné hodnocení plnění adaptačního plánu a evidenci hodnoticích rozhovorů	Must	R4, R6
F18	Systém poskytne přehled stavu adaptací podle závodu, oddělení a typu pracovní pozice	Should	R4, R6

Tabulka 2. 12: Funkcionální požadavky — Integrace a ověřování kvalifikací

ID	Požadavek	Priorita	Vazba
F20	Systém umožní automatizované ověření odborné způsobilosti pracovníků prostřednictvím API napojení na národní registr zdravotnických pracovníků	Should	R5
F21	Systém upozorní náboráře v případě, že uchazeč nemá platný záznam v NRZP nebo jeho způsobilost je omezena	Should	R5
F22	Systém eviduje výsledky ověření kvalifikací jako součást auditní stopy s časovým razítkem a identifikací zdroje ověření	Must	R3, R5

Tabulka 2. 13: Funkcionální požadavky — Reporting a multi-tenantní správa

ID	Požadavek	Priorita	Vazba
F23	Systém podporuje multi-tenantní model, kde každý odštěpný závod je samostatným tenantem s vlastními daty, uživateli a konfigurací	Must	R1
F24	Uživatelé s rolí centrálního administrátora mají přístup k datům a reportům napříč všemi tenanty	Must	R1, R6
F25	Systém poskytuje dashboards s klíčovými metrikami (počet otevřených pozic, průměrná doba obsazení, poměr přihlášek/přijetí, stav adaptací) na úrovni závodu i celé organizace	Should	R6
F26	Systém umožní export reportů do formátu PDF a CSV	Could	R6

2.5.2 NEFUNKCIONÁLNÍ POŽADAVKY

Nefunkcionální požadavky definují kvalitativní vlastnosti systému, které nejsou přímo pozorovatelné jako konkrétní funkce, ale podstatně ovlivňují použitelnost, spolehlivost a udržitelnost systému.

Tabulka 2. 14: Nefunkcionální požadavky na systém

ID	Oblast	Požadavek
NF01	Bezpečnost	Systém musí zajistit ochranu osobních údajů uchazečů a zaměstnanců v souladu s GDPR (nařízení 2016/679) a zákonem č. 110/2019 Sb. o zpracování osobních údajů
NF02	Bezpečnost	Autentizace uživatelů musí být zajištěna prostřednictvím protokolu OAuth 2.0 s integrací do existujícího SSO poskytovatele organizace
NF03	Bezpečnost	Systém musí implementovat přístup řízený rolemi (RBAC) s minimálně třemi úrovněmi — centrální administrátor, HR pracovník závodu, vedoucí oddělení
NF04	Dostupnost	Systém musí dosahovat dostupnosti alespoň 99,5 % v pracovních dnech v čase 6:00-22:00
NF05	Výkon	Odezva uživatelského rozhraní nesmí v 95. percentilu překročit 2 sekundy pro standardní operace (zobrazení seznamu, detail záznamu) při běžné zátěži
NF06	Přístupnost	Kariérní portál musí být responzivní, použitelný na mobilních zařízeních a navržený v souladu se zásadami WCAG 2.1 na úrovni AA
NF07	Nasaditelnost	Systém musí být nasaditelný na infrastruktuře organizace (on-premise) prostřednictvím kontejnerizace (Docker)
NF08	Udržitelnost	Zdrojový kód musí být verzován v systému pro správu verzí (Git) a dokumentován v rozsahu umožňujícím předání jinému vývojovému týmu
NF09	Lokalizace	Veškerá uživatelská rozhraní musí být v českém jazyce, systém musí správně pracovat s českou diakritikou ve všech vrstvách (databáze, API, UI)
NF10	Kompatibilita	Kariérní portál musí být kompatibilní s aktuálními verzemi prohlížečů Chrome, Firefox, Safari a Edge
NF11	Auditovatelnost	Systém musí uchovávat auditní záznamy o změnách klíčových entit (pozice, kandidát, adaptace) minimálně po dobu 5 let

3 EXISTUJÍCÍ SOFTWAREVÁ ŘEŠENÍ

Rozhodnutí o vlastním vývoji je obhajitelné jen tehdy, pokud je zřejmé, proč dostupné produkty nedokážou naplnit požadavky dané organizace bez nepřiměřených kompromisů. Rešeršní kapitola proto porovnává dostupná řešení z kategorií ATS (Applicant Tracking Systems), onboardingových platforem a integrovaných HR systémů a hodnotí je optikou podmínek KZ.

Při hodnocení jsou zohledněna zejména kritéria pokrytí požadavků na digitalizaci procesů uvedených v Tabulka 3. 8, české jazykové prostředí a lokalizace, přizpůsobitelnost procesům zdravotnické organizace a možnost provozování na vlastní infrastrukturu. Cílem není najít obecně nejznámější produkt na trhu, ale určit, zda některé z existujících řešení dokáže obhájit procesní, architektonické i provozní nároky této práce.

3.1 METODIKA REŠERŠE A HODNOTICÍ RÁMEC

Rešerši dostupných softwarových řešení jsem provedl jako strukturované komparativní posouzení produktů ze tří skupin. Jedná se o Applicant Tracking System (ATS), onboardingových platforem a integrovaných Human Capital Management (HCM) suite. Hodnocení vychází z veřejně dostupné produktové dokumentace výrobců, z oficiálních ceníků a z produktových stránek. Stav zdrojů odpovídá datu **21. 11. 2025**.

V této kapitole nehledám obecně nejlepší systém na trhu. Hledám řešení, které by bylo nejbližší podmínkám KZ, tedy zdravotnickému holdingu s požadavkem na on-premise provoz, auditovatelnost, multi-tenantní členění a návaznost na české procesy ověřování odborné způsobilosti zdravotnických pracovníků.

Tabulka 3. 1: Hodnoticí rámec řešerše dostupných řešení

ID	Hodnoticí kritérium	Vazba na požadavky	Typ
K1	Pokrytí náborového procesu (pozice, kandidáti, pipeline)	R2, R3	Must
K2	Pokrytí vstupní agendy a adaptace zaměstnance	R4	Must
K3	Podpora holdingového členění (oddělená data + centrální dohled)	R1	Must
K4	Integrace a rozšiřitelnost (API, datové vazby)	R5	Must
K5	Možnost provozu na vlastní infrastrukturu	NF07	Must
K6	Reporting a analytika napříč závody	R6	Should
K7	Lokalizace pro české prostředí	NF09	Should
K8	Podpora publikace inzerce na externích portálech	R2	Could

Pro účely rozhodnutí byla kritéria typu **Must** chápána jako diskvalifikační. Pokud řešení nesplní kterékoli kritérium K1-K5, není vhodné jako cílová platforma pro digitalizaci procesů KZ, i když může být přínosné jako dílčí integrační komponenta.

3.2 SPECIALIZOVANÉ ATS PLATFORMY

Specializované ATS nástroje jsou navrženy především pro akviziční část náboru. Silné bývají tam, kde je potřeba spravovat pozice, kandidátské workflow, komunikaci s uchazeči a publikaci pracovních nabídek. Otázkou pro tuto práci však není jen to, jak dobře umí nábor zahájit, ale zda dokážou přirozeně pokračovat do vstupní agendy a adaptační fáze, které jsou v prostředí zdravotnické organizace podstatně náročnější než v běžném komerčním náboru.

3.2.1 TEAMIO (ALMA CAREER)

Teamio lze považovat za referenční ATS řešení českého trhu, zejména díky jeho úzkému propojení s dominantními inzertními kanály Jobs.cz a Práce.cz. Tato vazba je v podmínkách KZ významná, protože umožňuje snížit transakční náklady spojené s publikací inzerce a konsolidací reakcí uchazečů. Oficiální do-

kumentace současně uvádí možnost automatického importu pozic z interních systémů i z portálů provozovaných Alma Career. [3]

Z hlediska funkčního pokrytí Teamio naplňuje klíčové ATS scénáře a v komerční nabídce deklaruje i preboarding/onboarding rozšíření. [4] Z pohledu požadavků této práce je však zásadní provozní model produktu, který je koncipován jako služba bez lokální instalace. [5] Tento model je sice vhodný pro rychlé uvedení do provozu, avšak nevytváří přímý soulad s požadavkem KZ na on-premise nasazení a interní kontrolu datového provozu.

Z analytického hlediska je proto Teamio vhodné vnímat spíše jako specializovanou integrační komponentu pro inzerci a sběr kandidátských reakcí než jako plnohodnotný cílový systém pro end-to-end řízení náboru, vstupní agendy a adaptace.

3.2.2 RECRUITIS

Recruitis je ATS platforma orientovaná na digitalizaci náborového workflow, práci s talent poolem a reportingovou podporu náborových aktivit. Veřejné materiály zároveň uvádějí vazbu na onboardingové řešení Onbee, což potvrzuje interoperabilitu systému, ale zároveň ukazuje, že náborová a adaptační vrstva jsou řešeny odděleně. [6]

Ve vztahu k enterprise požadavkům je důležitá deklarovaná podpora REST API integrace, Single Sign-On (SSO) a více právních subjektů. [6] Tyto parametry zvyšují použitelnost v heterogenním organizačním prostředí. Přesto i zde platí, že řešení komplexního adaptačního procesu by muselo být zajištěno navazujícím externím modulem, což zvyšuje závislost na mezisystémové koordinaci.

3.2.3 DATACRUIT

Datacruit deklaruje ATS funkcionalitu doplněnou o automatizační prvky a integrační otevřenost. [7] Z hlediska této práce je klíčové, že oficiální ceník u enterprise varianty uvádí možnost implementace na vlastní infrastrukturu zákazníka a licenční model, který není čistě subscription-based. [8]

Tento parametr činí Datacruit z porovnávaných ATS systémů nejbližším kandidátem ve vztahu k požadavku NF07. Funkční těžiště produktu je však nadále v oblasti náboru. Plnohodnotná podpora adaptačního procesu zdravotnických pracovníků by tedy vyžadovala další produktovou vrstvu nebo zakázkové rozšíření, což zvyšuje architektonickou i provozní složitost cílového řešení.

3.2.4 SLONEEK (ATS MODUL)

Sloneek nabízí ATS modul jako součást širší HR platformy. [9] Tento přístup je výhodný z pohledu sjednocení základních personálních agend, avšak dostupná

dokumentace současně uvádí omezení v oblasti přímé integrace ATS modulu na externí platformy. [9]

V prostředí KZ tak Sloneek představuje využitelnou variantu pro obecnou digitalizaci HR činností, nikoli však jednoznačně vhodného kandidáta pro cílové řešení s požadovanou mírou specializace na zdravotnické procesy, multi-tenantní řízení a on-premise provoz.

3.3 ONBOARDINGOVÉ PLATFORMY

3.3.1 ONBEE

Onbee představuje specializovanou onboarding/offboarding platformu zaměřenou na procesní koordinaci nástupu zaměstnance, práci s checklisty a správu související dokumentace. Ve veřejné produktové dokumentaci jsou deklarovány integrační možnosti na ATS/HRIS, podpora SSO i možnost on-premise instalace. [10]

Z hlediska hodnoticího rámce tak Onbee velmi dobře pokrývá část požadavků spojených s adaptací (R4) a částečně i provozní požadavky NF07. Jeho limit spočívá v tom, že nejde o plnohodnotné ATS řešení, takže by organizace musela paralelně provozovat a integračně udržovat další náborový systém. Vznikla by tak vícevrstvá architektura s vyšší závislostí na kvalitě integračních vazeb.

3.4 INTEGROVANÉ HCM SUITE (ENTERPRISE)

Integrované HCM platformy sledují jiný strategický cíl než specializované ATS nebo onboardingové nástroje. Snaží se pokrýt co největší část zaměstnaneckého cyklu v jednom datovém a procesním modelu. Z pohledu enterprise architektury jde o logický přístup, protože snižuje fragmentaci dat. V praxi však bývá vykoupen vyšší implementační náročností, větším rozsahem změnového řízení a velmi často i cloudovým provozním modelem.

3.4.1 SAP SUCCESSFACTORS

SAP SuccessFactors je výrobcem prezentován jako cloud-based HCM suite s moduly pro recruiting i onboarding. [11, 12] Z hlediska funkční šíře jde o robustní platformu schopnou pokrýt velkou část HR procesů, včetně centralizovaného reportingu a governance.

V podmínkách KZ je však hlavním limitem provozní model orientovaný na cloud a nutnost rozsáhlé konfigurace na lokální zdravotnické specifikum. Rizikem je

i vyšší implementační setrvačnost při změnách procesního modelu během projektu.

3.4.2 WORKDAY

Workday deklaruje jednotnou cloud platformu pro HR a související procesy včetně talent acquisition. [13, 14] Ve srovnání s tradičním modulárním přístupem poskytuje silný důraz na datovou konzistenci a analytiku napříč procesy.

Stejně jako u SAP SuccessFactors je však v této práci rozhodující nesoulad mezi cloudovým provozním modelem a požadavkem KZ na provoz na vlastní infrastrukturu. Dalším faktorem je náročnost lokalizace na české regulační prostředí a specifika zdravotnických rolí.

3.4.3 ORACLE FUSION CLOUD HCM

Oracle Fusion Cloud HCM je výrobcem prezentován jako kompletní cloudové řešení zahrnující recruiting i onboarding procesy. [15, 16] Z hlediska procesního pokrytí jde o srovnatelně široký koncept jako u předchozích enterprise platform.

V případě KZ však i zde převažuje stejná bariéra: nesoulad s požadavkem na on-premise provoz a vysoká míra přizpůsobení nutná pro české zdravotnické procesy, zejména ve vazbě na národní registry a interní organizační členění holdingu.

3.5 POROVNÁNÍ ŘEŠENÍ VŮČI POŽADAVKŮM KZ

Následující tabulky shrnují vybraná řešení podle diskvalifikačních kritérií K1-K5 a doplňkových kritérií K6-K8. Hodnocení je provedeno kvalitativně ve škále **Ano / Částečně / Ne** na základě veřejně doložitelných informací.

Pro Tabulka 3. 2 platí: K1 = nábor, K2 = adaptace, K3 = multi-tenantní podpora, K4 = integrace/API, K5 = on-premise provoz.

Tabulka 3. 2: Porovnání vybraných řešení podle diskvalifikačních kritérií

Řešení	K1	K2	K3	K4	K5
Teamio	Ano	Částečně	Částečně	Ano	Ne
Recruitis	Ano	Částečně	Částečně	Ano	Ne
Datacruit	Ano	Částečně	Částečně	Ano	Ano
Sloneek	Částečně	Částečně	Částečně	Částečně	Ne
Onbee	Ne	Ano	Částečně	Ano	Ano
SAP SuccessFactors	Ano	Ano	Ano	Ano	Ne
Workday	Ano	Ano	Ano	Ano	Ne
Oracle HCM	Ano	Ano	Ano	Ano	Ne

Tabulka 3. 3: Porovnání vybraných řešení podle doplňkových kritérií K6-K8

Řešení	K6 Reporting	K7 Lokalizace	K8 Inzerce
Teamio	Ano	Ano	Ano
Recruitis	Ano	Ano	Ano
Datacruit	Ano	Částečně	Ano
Sloneek	Ano	Ano	Částečně
Onbee	Částečně	Ano	Ne
SAP SuccessFactors	Ano	Částečně	Částečně
Workday	Ano	Částečně	Částečně
Oracle HCM	Ano	Částečně	Částečně

Interpretace výsledků v Tabulka 3. 2 ukazuje, že žádný z hodnocených produktů nesplňuje současně všechna diskvalifikační kritéria K1-K5. Enterprise HCM platformy dosahují vysokého funkčního pokrytí nábory i adaptace, avšak jejich cloudová orientace je v přímém rozporu s provozními omezeními KZ. Specializované ATS platformy naopak přinášejí vysokou efektivitu v akviziční části nábory, nicméně navazující adaptační proces obvykle pokrývají jen částečně nebo prostřednictvím odděleného nástroje.

Z porovnání doplňkových kritérií vyplývá, že rozdíly mezi produkty se zmenšují v oblastech reportingu a obecné lokalizace, které jsou v komerčních řešeních obvykle dobře pokryty. Rozhodující je proto schopnost naplnit specifické požadavky domény zdravotnictví a interní architektonická omezení organizace, nikoli pouze šíře standardních HR funkcí.

Specifickým požadavkem zdravotnického prostředí je napojení na ověřování odborné způsobilosti zdravotnických pracovníků (NRZP), které je v českém eGovernment ekosystému dostupné jako samostatná služba. [17] Ve veřejné

dokumentaci hodnocených komerčních produktů nebyla k datu rešerše doložena explicitní, hotová podpora tohoto procesu bez doplňkového vývoje nebo integrační nadstavby.

3.6 IDENTIFIKOVANÁ OMEZENÍ DOSTUPNÝCH PRODUKTŮ A VOLBA CÍLOVÉHO PŘÍSTUPU

Zjištění ukazují, že hlavní limit dostupných produktů neleží v absenci jednotlivých funkcí. Leží v nesouladu mezi jejich produktovou logikou a cílovou architekturou KZ. První omezení představuje provozní model. Významná část funkčně robustních enterprise platform je koncipována jako cloud-native služba, zatímco KZ vyžaduje provoz na vlastní infrastruktuře z důvodu interní governance, bezpečnostních politik a provozní autonomie.

Druhé omezení se týká procesní kontinuity. U specializovaných ATS řešení je často vysoká kvalita náborové části, avšak adaptační proces bývá realizován v odděleném nástroji. Vzniká tak architektura „best of breed“, která je funkčně flexibilní, ale z pohledu provozu přináší rizika fragmentace dat, složitější správu identit a vyšší náklady na dlouhodobé integrační testování.

Třetí omezení spočívá v doménové specifitě českého zdravotnictví. Hodnocené produkty jsou navrženy jako obecně použitelné HR platformy pro více odvětví a zemí. To je jejich komerční výhoda, ale současně limit v situaci, kdy organizace potřebuje cíleně implementovat lokální procesní logiku, například ověřování odborné způsobilosti, auditovatelný průchod zdravotnickou adaptací nebo detailní vazbu na interní předpisy jednotlivých závodů.

Čtvrtým omezením je dlouhodobá provozní závislost na produktových plánech dodavatelů. U klíčových procesů, které jsou pro KZ strategické, představuje tato závislost významné riziko. Organizace by musela adaptovat interní postupy podle vývojového směru třetí strany, což je v rozporu s cílem řídit transformaci procesů primárně podle vlastních potřeb.

Na základě uvedených zjištění jsem zvolil jako nejvhodnější přístup založený na vývoji vlastního řešení. Tento přístup mi umožňuje navrhnout jednotný datový model pro nábor, vstupní agendu i adaptaci, implementovat multi-tenantní architekturu odpovídající holdingovému uspořádání a současně respektovat požadavek na on-premise provoz bez kompromisních výjimek.

Vlastní řešení zároveň snižuje riziko, že kritické doménové požadavky budou odloženy kvůli externí produktovému plánu. Integrace, které jsou pro KZ skutečně přínosné, lze realizovat selektivně a v řízeném rozsahu, například pro distribuci inzerce nebo výměnu dat s externími registry. Tím se kombinuje výhoda procesní suverenity s pragmatickým využitím existujícího softwarového ekosystému.

Z metodického hlediska tedy tato kapitola nevede k závěru, že komerční produkty jsou obecně nevhodné. Vede k závěru, že pro konkrétní kombinaci požadavků R1-R6 a NF01-NF12 je v podmínkách KZ vlastní řešení variantou s nejvyšší mírou strategické a provozní shody.

4 NÁVRH SOFTWAREVÉ ARCHITEKTURY

Architektura navrženého řešení musela odpovědět na praktickou otázku, jak digitalizovat nábor a adaptaci v prostředí, které je současně organizačně členité, regulatorně citlivé a provozně omezené on-premise infrastrukturou. Nešlo tedy o návrh obecně elegantního systému, ale o takovou architekturu, která bude obhajitelná vůči požadavkům Krajské zdravotní, a.s. a zároveň zůstane dlouhodobě udržitelná.

4.1 ARCHITEKTONICKÉ VÝCHODISKO A VOLBA CÍLOVÉHO STYLU

Architektonická rozhodnutí jsem odvozoval přímo z provozního kontextu KZ. Organizace není startup na zelené louce s jedním produktem a jedním týmem. Jde o holding se sedmi odštěpnými závody, regulovanými daty, heterogenními uživatelskými rolemi a omezenými provozními kapacitami. Z toho plyne, že architektura musí být nejen technicky správná, ale i srozumitelná pro další rozvoj a bezpečně provozovatelná.

Nejsilnějším strukturálním požadavkem je multi-tenantní izolace dat (R1). Každý závod musí pracovat se svými daty, ale centrální vedení potřebuje průřezový pohled. Tento požadavek proto nelze řešit jen na úrovni oprávnění v uživatelském rozhraní. Musí být propsán do datového modelu, aplikačních služeb i do způsobu, jakým se vyhodnocují přístupová pravidla. Vedle toho musí systém pokrýt celý tok od zveřejnění pozice přes nábor a vstupní agendu až po adaptaci zaměstnance (R2-R4), být připraven na napojení externích služeb (R5) a poskytovat data pro řízení procesu (R6).

Z kvalitativních atributů [18] jsem kladl největší důraz na bezpečnost, auditovatelnost a provozní udržitelnost. V prostředí zdravotnictví není auditní stopa doplňkem, ale základní podmínkou důvěryhodnosti systému. Stejně tak on-premise provoz znamená, že nemohu spoléhat na komfort cloudových managed služeb. Architektura proto musí minimalizovat zbytečnou distribuovanou složitost a současně připustit řízený růst tam, kde je skutečně potřebný.

Při volbě architektonického stylu jsem porovnával čtyři varianty: klasický monolit, modulární monolit, přístup microservices-first a hybridní model. Rozhodnutí jsem nevázal na technologickou módnost, ale na to, jak dobře daný přístup unese

požadavky na modularitu, integrace, provozní jednoduchost a budoucí rozšiřitelnost.

Tabulka 4. 1: Porovnání architektonických variant

Varianta	Silné stránky	Rizika
Klasický monolit	Nejnižší počáteční složitost a rychlé zavedení v malé fázi	Slabé architektonické hranice, horší dlouhodobá udržitelnost a omezená evoluce integračních scénářů
Modulární monolit	Nízká integrační režie, konzistentní doménový model, rychlejší implementace	Riziko růstu vnitřní vazby při nekázni modulárních hranic
Microservices-first	Vysoká autonomnost částí, nezávislé škálování	Vysoká distribuovaná komplexita a provozní režie pro aktuální fázi projektu
Hybrid	DDD modulární jádro + separované procesory vrstvy inteligentního zpracování dat	Nutnost řídit integrační kontrakty mezi jádrem a procesory

Klasický monolit by sice zjednodušil první implementační kroky, ale rychle by rozostřil hranice mezi náborovou logikou, integracemi a analytickými funkcemi. Přístup microservices-first by naopak od začátku vnesl síťovou komunikaci, distribuované selhávání a vyšší provozní režii do projektu, který je vyvíjen iterativně a s omezenými kapacitami [19]. Samotný modulární monolit proto vycházel jako nejpraktičtější základ, ale neřešil dobře oddělení výpočetně náročných částí spojených s inteligentním zpracováním dat.

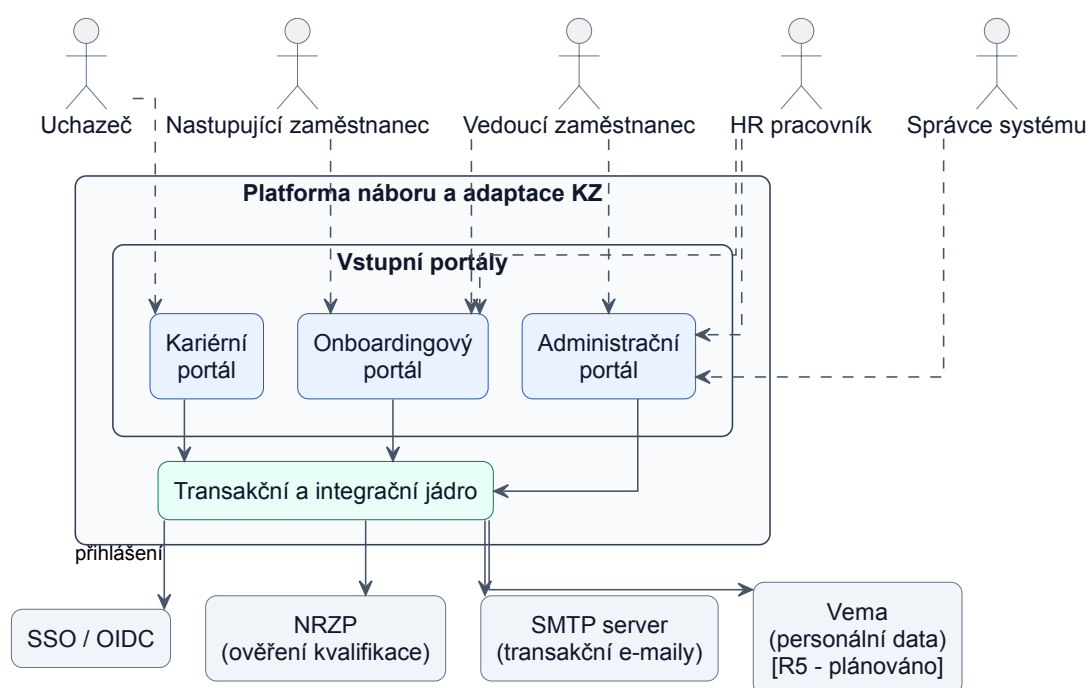
Jako cílový styl jsem proto zvolil hybridní architekturu. Transakční jádro systému jsem navrhl jako modulární monolit s využitím principů Domain-Driven Design a Ports and Adapters, zatímco vrstvu inteligentního zpracování dat jsem oddělil do specializovaných komponent `cv_processor` a `job_processor`. Výsledkem je kompromis, který zachovává jednoduchost hlavního jádra, ale nepřenáší výpočetní zátěž a integrační rizika do kritické transakční cesty.

Další výklad proto neorganizuji podle jedné univerzální diagramové taxonomie, ale podle otázek, na které musí výsledná architektura odpovědět. Nejprve

ukazují celkový obraz řešení, poté vnitřní strukturu backendu, následně datové a integrační vazby a nakonec bezpečnostní a provozní důsledky zvoleného návrhu. Takový postup lépe odpovídá tomu, co je v této práci skutečně potřeba obhájit.

4.2 CELKOVÝ OBRAZ ŘEŠENÍ

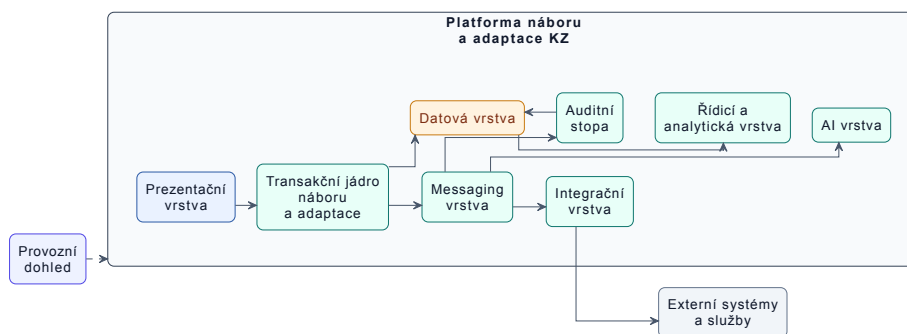
Prvním krokem je zachytit systém v kontextu rolí a externích služeb. Tím se vyjasní, proč řešení nevzniklo jako jedna univerzální interní aplikace, ale jako soustava více vstupních bodů nad společným transakčním a integračním jádrem.



Obrázek 4. 1: Systém v kontextu rolí a externích služeb

Z Obrázek 4. 1 je patrné, že systém obsluhuje několik odlišných skupin uživatelů s různými cíli a různou mírou oprávnění. Uchazeči vstupují přes veřejný kariérní portál, interní náborová agenda je soustředěna do administračního portálu a nástup se sledováním adaptace do onboardingového portálu nad stejným doménovým jádrem. Na straně externích vazeb jsou rozhodující SSO/OIDC, NRZP a e-mailová infrastruktura. Právě tyto závislosti potvrzují, že řešení musí být navrženo jako integrační uzel, nikoli jen jako izolovaná evidence.

Na kontextový pohled navazuje logická kompozice řešení. Jejím smyslem není popsat konkrétní nasazovací topologii ani technologickou skladbu, ale ukázat, z jakých stavebních bloků se systém skládá a proč jsou hranice mezi nimi vedeny právě tímto způsobem.



Obrázek 4. 2: Logická kompozice řešení

Systém musí současně zvládat transakční agendu náboru a adaptace, komunikaci s okolními službami, výpočetně náročné zpracování i průběžné vyhodnocování procesu. Pokud by se tyto odpovědnosti soustředily do jedné vrstvy, docházelo by ke míchání stabilní domény s proměnlivými integračními a analytickými požadavky, což by vedlo ke ztrátě přehlednosti i rozšiřitelnosti.

Na tuto situaci odpovídá Obrázek 4. 2. V centru návrhu stojí transakční jádro náboru a adaptace, které nese business pravidla a hlavní procesní tok. K němu se připojuje prezentační vrstva obsluhující veřejný kariérní portál i interní rozhraní, datová vrstva uchováající stav procesu, samostatná messaging vrstva pro asynchronní předávání úloh a událostí, integrační vrstva zajišťující řízený kontakt s okolím, auditní stopa pro dohledatelnost změn a oddělená AI vrstva pro výpočetně náročné zpracování. Diagram záměrně abstrahuje od konkrétních technologií, protože jejich realizační rozpad patří až do implementační kapitoly. Na architektonické úrovni je důležité obhájit logiku hranic, ne vyjmenovat každou nasazenou službu.

Návrh vychází přímo z požadavků R1–R6. Požadavky R2–R4 definují tři navazující situace (veřejný vstup, interní řízení, adaptace), což vede k oddělení prezentační vrstvy od jednotného transakčního jádra. Požadavek R1 pak promítá holdingovou strukturu KZ do datové vrstvy, která tak nese nejen stav systému, ale i organizační kontext.

Oddělení řídicí a analytické vrstvy je klíčové. Systém nemá pouze evidovat průběh náboru a adaptace, ale také poskytovat podklady pro jeho řízení a průběžné zlepšování. Proto do této vrstvy patří nejen vyhodnocování průchodnosti náboru a adaptace, ale i analytika kariérního portálu. Ta umožňuje sledovat, jak se potenciální uchazeči na portálu chovají, kde ztrácejí pozornost, ve kterých krocích proces opouštějí a které části nabídky nebo formulářů snižují konverzi. Právě proto je tato vrstva oddělena od technického provozního dohledu. Provozní dohled odpovídá na otázku, zda systém běží zdravě, zatímco řídicí a analytická vrstva vysvětluje, co se v náborovém procesu skutečně děje.

Samostatné zachycení auditní stopy je přímou odpovědí na problém P4, tedy absenci auditní stopy v původním procesu, i na nefunkcionální požadavek NF11. V logické kompozici ji proto nevedu jako další byznysovou oblast, ale jako pod-

půrnou architektonickou schopnost systému. Auditní událost vzniká v jádře, je předána přes messaging a následně perzistována do datové vrstvy. Tím zůstává audit mimo hlavní cestu požadavků a současně je zachována dohledatelnost změn nad klíčovými entitami.

Stejně podstatná je integrační vrstva. Její role nespočívá jen v napojení na NRZP, které přímo vychází z požadavku R5. Stejně důležitá je schopnost bezpečně připojovat i další okolní systémy a služby, které se promítají do každodenního provozu náboru a adaptace, například identitní služby, VEMA, národní registry nebo komunikační infrastrukturu. Nejde tedy o jednu konkrétní integraci, ale o řízenou hranici mezi transakčním jádrem a cizími systémy. Transakční jádro zde pouze vyjadřuje, jakou schopnost od okolí potřebuje, například vyhledat kvalifikaci, interního uživatele nebo ověřit identitu, zatímco integrační vrstva přebírá odpovědnost za cizí protokol, datový model i chybové stavy externí služby. Smyslem této vrstvy je převzít integrační složitost na sebe, aby se nepropisovala přímo do transakční logiky.

Vedle ní stojí messaging vrstva, která neřeší, na jaký externí systém se systém napojuje, ale jak jsou úlohy a události předávány mimo hlavní transakční tok.

4.3 STRUKTURA BACKENDU

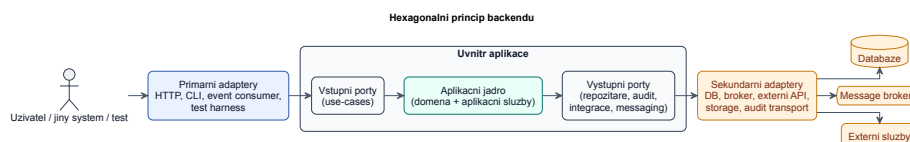
Nejcitlivější částí celého řešení je backend, protože právě v něm se setkávají business pravidla, datové hranice, bezpečnost i integrace. Jádro backendu proto stavím na hexagonální architektuře, kterou Cockburn popsal jako vzor Ports and Adapters [20]. Motivací tohoto vzoru je oddělit aplikační logiku od uživatelského rozhraní, databáze a dalších zařízení tak, aby systém bylo možné testovat, provozovat i rozvíjet bez přímé závislosti na konkrétní technologii. V praktickém smyslu to znamená, že business pravidla nesmějí být uzamčena v controlleru, HTTP frameworku ani databázovém driveru.

Klíčová asymetrie této architektury neleží mezi „horní“ prezentační a „spodní“ datovou vrstvou, ale mezi vnitřkem a vnějškem aplikace [20]. Port v tomto pojetí nepředstavuje síťový port, nýbrž účelově definovaný kontrakt komunikace mezi jádrem a jeho okolím. K jednomu portu přitom může existovat více adaptérů, například produkční HTTP vstup, konzolový přístup nebo náhradní implementace persistence pro integrační testy [20]. Tento přístup je důležitý ze tří důvodů. Zaprvé udržuje multi-tenantní a organizační kontext pod kontrolou od vstupu do systému až po datovou vrstvu. Zadruhé chrání doménu před přímou závislostí na integračních detailech, které se mohou v čase měnit. Zatřetí vytváří čitelnou strukturu, kterou lze dlouhodobě rozvíjet i bez velkého specializovaného architektonického týmu.

V terminologii této práce proto označuji adaptéry, které aplikaci řídí zvenku dovnitř, jako primární adaptéry, zatímco adaptéry vykonávající požadavky jádra směrem do infrastruktury označuji jako sekundární adaptéry. Primární adaptér

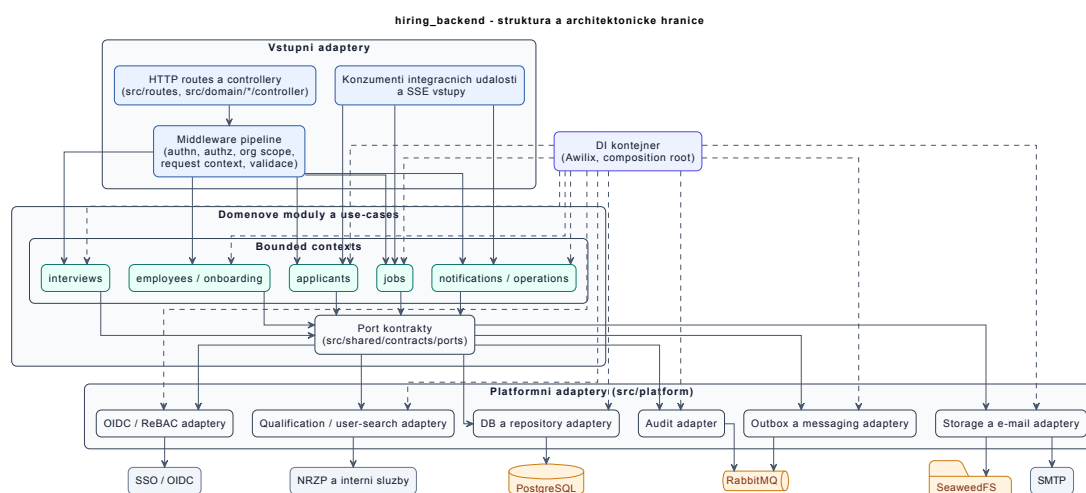
převádí vnější signál na volání use-casu, kdežto sekundární adaptér implementuje port definovaný jádrem a překládá jeho požadavek do konkrétního protokolu, API nebo perzistenční technologie. Současně je důležité odlišit architektonickou hranici od hranice nasazovací. Hexagonální architektura sama o sobě neznamená, že každý adaptér musí být samostatná služba. Naopak většina adaptérů běží uvnitř jednoho backendového procesu a samostatně oddělují jen ty části, jejichž runtime, provozní profil nebo integrační režim se od jádra skutečně liší.

Hexagonální uspořádání v mém návrhu současně doplňuje principy Domain-Driven Design. Doménové moduly jsou vedeny jako ohraničené kontexty s vlastním modelem a slovníkem, protože význam pojmů je v doménovém návrhu vždy platný jen uvnitř explicitně vymezené hranice [21, 22]. Na styku s externími službami proto integrační vrstva plní roli překladové mezivrstvy. Doména pracuje se svým modelem, zatímco adaptér přebírá odpovědnost za převod z cizího rozhraní a zpět [22].



Obrázek 4. 3: Obecný princip hexagonální architektury

Obecný princip zachycený na Obrázek 4. 3 se v hiring_backend promítá do konkrétní struktury backendu. Primární adaptéry na vstupu aplikace převádějí HTTP požadavky a integrační události na use-cases domény. Uvnitř aplikace zůstává doménová vrstva členěná do ohraničených kontextů a komunikuje s okolím pouze prostřednictvím explicitních port kontraktů. Sekundární adaptéry pak zajišťují napojení na databázi, messaging, audit, autorizaci a externí registry.



Obrázek 4. 4: Návrh hranic backendu v hexagonálním uspořádání

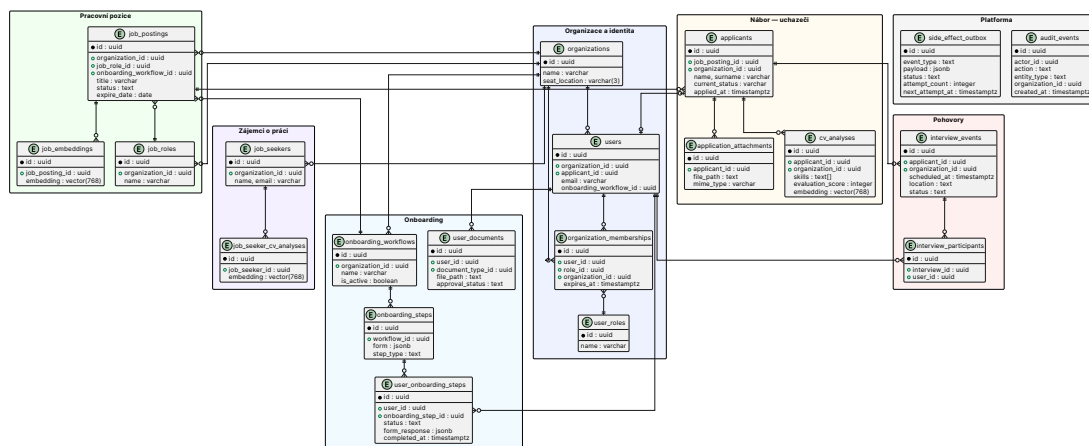
Obrázek 4. 4 už neslouží jako inventář implementačních složek, ale jako návrh hlavních hranic backendu. Ukazuje, že vstupní adaptéry řídí vstup do systému, doménové jádro nese use-cases a ohraničené kontexty, porty vymezují povolené závislosti a sekundární adaptéry připojují databázi, messaging, audit, bezpečnost a okolní služby. Smyslem tohoto rozdělení je udržet stabilnější business logiku uvnitř a proměnlivější integrační infrastrukturu na hraně systému.

4.4 DATOVÝ MODEL A INTEGRAČNÍ TOKY

Architektonická rozhodnutí by zůstala neúplná, pokud by se nepropsala i do datového modelu. Ten tvoří přibližně šedesát tabulek organizovaných do sedmi doménových skupin. Cílem této části proto není vypsát celé schéma, ale vysvětlit, jaké logické celky model obsahuje a proč je navržen právě tímto způsobem. Technické detaily migrací a fyzické perzistence rozvádím v následující implementační kapitole.

Základ datového modelu tvoří organizace a návazné členství interních uživatelů. Právě tato skupina entit nese organizační kontext, bez něhož by nebylo možné bezpečně oddělit data jednotlivých závodů a současně zachovat centrální reporting. Další oblast tvoří model pracovních pozic, který propojuje klasifikaci rolí, typy úvazků, vazbu na inzerát i návaznost na onboardingové workflow. Pozice zde není pouze text inzerátu, ale procesní uzel, kolem něhož se soustředují kandidáti, pohovory, dokumenty a odpovědnosti.

Náborová oblast pokrývá uchazeče, jejich přihlášky, přiložené dokumenty a výstupy kvalifikačních nebo analytických kroků. Na ni navazuje pohovorová agenda, která propojuje kandidáta, interní účastníky a organizační kontext. Onboardingová oblast pak modeluje workflow nástupu, jednotlivé kroky, dokumenty a jejich stav plnění. Samostatnou skupinu tvoří zájemci o práci bez vazby na konkrétní pozici a poslední vrstvu představují průřezové záznamy podporující auditovatelnost a provozní spolehlivost, zejména evidence změn, idempotence a asynchronních vedlejších efektů.



Obrázek 4. 5: Konceptuální ER diagram hlavních entit a jejich vztahů

Obrázek 4. 5 zachycuje hlavní entity sedmi doménových skupin a jejich vzájemné vazby. Z důvodu rozsahu (přibližně 60 tabulek) nejsou zobrazeny všechny referenční číselníky, pomocné indexační tabulky ani historické záznamy stavů. Diagram slouží především k tomu, aby bylo vidět, že nábor, vstupní agenda a adaptace nejsou v návrhu oddělené ostrovy, ale propojené části jednoho datového modelu.

Prvním důležitým rozhodnutím je použití flexibilní JSON struktury pro onboardingové formuláře a odpovědi zaměstnanců. Zvolil jsem ji proto, že tato část procesu se může průběžně měnit podle role, pracoviště i interních pravidel. Pevně relační model by zde vedl k častým schématickým změnám i kvůli drobným úpravám obsahu formulářů.

Druhým rozhodnutím je využití pgvector přímo v databázi PostgreSQL. Embeddingy životopisů a pracovních pozic tak lze ukládat vedle transakčních dat a vyhledávání zůstává součástí jednoho technologického celku. Nevolím tedy samostatné vektorové úložiště, protože by v této fázi projektu přineslo více integrační složitosti než praktického přínosu.

Třetím rozhodnutím je explicitní vazba mezi uchazečem a vzniklým zaměstnancem. Tato návaznost umožňuje dohledat celý proces od přihlášky přes nábor až po onboarding. V prostředí, kde je důležitá auditní stopa a vyhodnocování adaptačních výsledků, jde o zásadní vlastnost modelu.

Čtvrtým rozhodnutím je důsledné nesení atributu `organization_id` u klíčových entit. Multi-tenantní charakter systému nesmí být jen pravidlem v dokumentaci, ale musí být fyzicky přítomen v datech a následně respektován v aplikační logice.

Na datový model přímo navazuje způsob, jakým systém komunikuje s okolními službami a jak zpracovává vedlejší efekty. Integrační vrstvu proto navrhuji kombinací synchronní a asynchronní komunikace. Synchronní volání používám tam, kde uživatel očekává okamžitou odezvu, například při práci s administračním rozhraním nebo při generování konkrétního výstupu. Asynchronní komunikaci

naopak využívám pro vedlejší efekty a časově náročnější operace, u nichž by blokování requestu zhoršovalo použitelnost systému. Konkrétní realizační tok této kombinace, včetně využití vzoru transactional outbox a návaznosti na vrstvu inteligentního zpracování dat, uvádím až v implementační kapitole.

Pro komunikaci směrem ke klientovi architektura počítá s mechanismem (SSE) průběžného informování o stavu operací. Ten umožňuje zobrazovat změny nebo postup zpracování bez nutnosti opakovaného dotazování klienta a zároveň nezatěžuje systém plně obousměrnou komunikací.

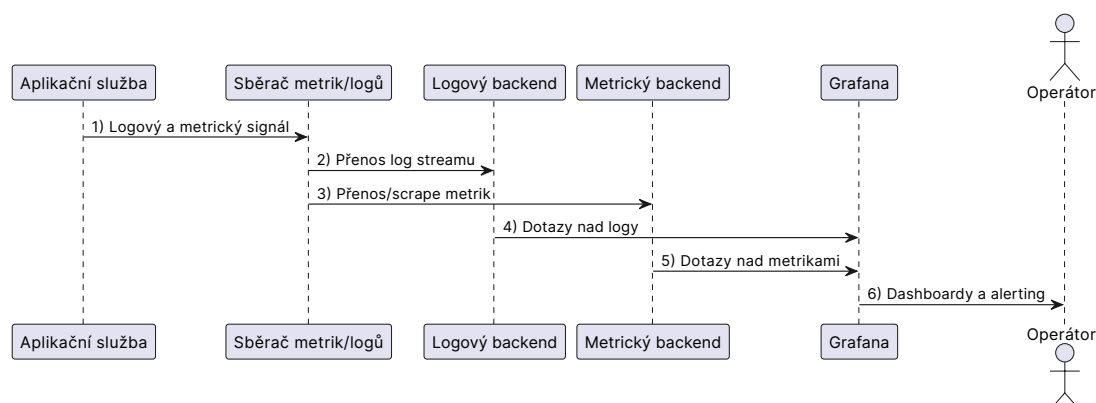
4.5 BEZPEČNOST A PROVOZNÍ DOHLED

Jakmile systém propojuje více rolí, více závodů a citlivé personální údaje, stává se bezpečnostní architektura jedním z rozhodujících kritérií návrhu. Proto ji navrhuji ve třech vrstvách: autentizace, autorizace a datový rozsah. Autentizace je řešena prostřednictvím OIDC/SSO, aby interní uživatelé nemuseli spravovat oddělené identity pouze pro tento systém. Tento krok zároveň snižuje provozní režii spojenou se správou účtů.

Autorizaci nestavím jen na klasickém RBAC, ale na kombinaci globálních rolí a ReBAC (relationship-based access control). Vedl mě k tomu proces, kde vedoucí pracovník může potřebovat přístup jen k jedné konkrétní pozici nebo skupině kandidátů, nikoli k celému závodu. Samotná role tak popisuje typ uživatele, zatímco vztah ke zdroji rozhoduje o skutečném rozsahu dat.

Součástí bezpečnostního návrhu je i princip nejnižších oprávnění a auditovatelnost operací nad citlivými daty (NF11). Změny nad klíčovými entitami musí být zpětně dohledatelné, a to nejen kvůli bezpečnosti, ale i kvůli schopnosti vysvětlit průběh náboru nebo adaptace při interním přezkumu.

Bezpečnostní vrstva však sama nestačí. V on-premise prostředí je stejně důležité včas rozpoznat, že se systém začíná odchylovat od očekávaného chování. Proto architekturu uzavírá samostatná vrstva provozního dohledu. Kombinace Promtail, Loki, Prometheus a Grafana umožňuje sběr logů, metrik i jejich společnou interpretaci z jednoho místa, aniž by bylo nutné zasahovat do business logiky systému.



Obrázek 4. 6: Tok dat v monitorovací vrstvě od aplikace po vizualizaci

Vedle samotného sběru dat navrhuji i dva typy alertovacích mechanismů. Provozní alerty mají zachytit zhoršení dostupnosti nebo latencí klíčových endpointů. SLO alerty pak sledují zdraví asynchronní vrstvy, například stáří nejstarší nevyřízené zprávy nebo počet zpráv, které selhaly po maximálním počtu pokusů. Smyslem této vrstvy není produkovat více dashboardů, ale dát provozu včasný signál, že se systém začíná odchylovat od očekávaného chování.

5 IMPLEMENTACE SYSTÉMU

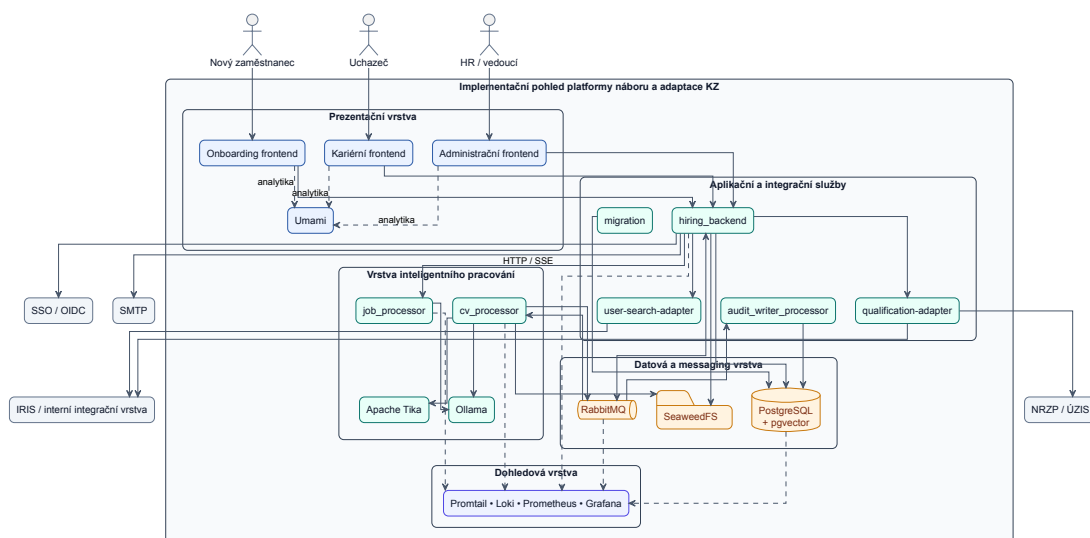
Architektonický návrh má smysl jen tehdy, pokud se podaří převést jeho principy do konkrétní implementace bez toho, aby se po cestě rozpadly pod tlakem technologických kompromisů. V této kapitole proto nesleduji úplný výpis všech tříd, tabulek a endpointů, ale ta rozhodnutí, na nichž se láme rozdíl mezi formálně správným návrhem a skutečně provozně použitelným systémem.

Pozornost soustředím především na modulární backend, hexagonální rozdělení odpovědností, spolehlivou asynchronní komunikaci, datovou vrstvu, bezpečnostní model a oddělenou vrstvu inteligentního zpracování dat. Právě v těchto částech se ukazuje, zda navržená architektura dokáže unést reálné procesní a provozní požadavky KZ.

5.1 STRUKTURA ŘEŠENÍ

Celé řešení je implementováno jako sada spolupracujících aplikací a služeb s oddělenými odpovědnostmi. Transakční část systému je realizována v prostředí Node.js/Express, webové portály jsou postaveny na Next.js a podpůrné zpracovatelské či integrační služby běží převážně v jazyce Go. Toto rozdělení nevyplývá z technologické preference samo o sobě, ale z rozdílného charakteru jednotlivých problémů. Backend vyžaduje doménovou konzistenci, frontendy rychlý rozvoj rozhraní a procesory vrstvy inteligentního zpracování dat efektivní práci s asynchronními úlohami.

Vedle hlavního backendu tak řešení zahrnuje i samostatné služby pro auditní zápis, integrační adaptéry a inteligentní zpracování dokumentů či textů pracovních pozic. Tyto komponenty spolu komunikují přes HTTP rozhraní nebo přes RabbitMQ. Praktickým důsledkem je možnost odděleně nasazovat a škálovat části s odlišným provozním profilem, aniž by se celý systém rozpadl na síťově fragmentovaný soubor mikroslužeb.



Obrázek 5. 1: Diagram implementace

5.2 IMPLEMENTACE BACKENDU

Backend jsem implementoval jako modulární aplikaci s dominantním hexagonálním uspořádáním. Každý požadavek vstupuje přes route a middleware pipeline, která řeší autentizaci, autorizaci, request kontext a mapování chyb. Teprve potom přechází řízení do doménové služby, kde probíhá business logika.

Z fyzického hlediska je backend rozdělen do pěti hlavních částí. `src/routes` a `src/app.js` tvoří vstupní HTTP adaptéry. `src/domain` obsahuje byznysové moduly, například `jobs`, `applicants`, `employees`, `qualification` nebo `internalUsers`. `src/shared/contracts/ports` drží explicitní kontrakty a `src/platform` soustřeďuje technologické adaptéry pro databázi, audit, messaging, storage, autentizaci, ReBAC i interní integrační služby. Vazby mezi těmito částmi spravuje dependency injection kontejner `Awilix`.

Tabulka 5. 1: Implementační realizace hexagonální architektury backendu

Prvek hexagonu	Implementační realizace
Vstupní adaptéry	<code>src/routes</code> , kontrolery v <code>src/domain/*/controller</code> , middleware a bootstrap v <code>src/app.js</code>
Doménové moduly	<code>src/domain/*/service</code> , <code>repository</code> , <code>events</code> a modulové registrace přes <code>index.js</code>
Porty	<code>src/shared/contracts/ports</code> , například <code>cvPublishPort</code> , <code>internalUsersPort</code> , <code>rebacPort</code>
Výstupní adaptéry	<code>src/platform/*</code> , například <code>platform/qualification</code> , <code>platform/userSearch</code> , <code>platform/audit</code> , <code>platform/outbox</code> , <code>platform/storage</code>
Vazba port-adaptér	DI registr <code>src/container.registry.js</code> a Awilix tokeny

5.2.1 POUŽITÉ NÁVRHOVÉ A INTEGRAČNÍ VZORY V `HIRING_BACKEND`

Architektonická kapitola obhájí principy backendu na koncepční úrovni. Z pohledu implementace je ale důležité ukázat, že nejde o abstraktní slovník pojmů, nýbrž o soubor konkrétních vzorů, které společně drží `hiring_backend` čitelný, rozšiřitelný a provozně stabilní.

Nejvýrazněji je přítomen vzor `Ports and Adapters`, tedy hexagonální architektura. Řeší problém, jak oddělit doménovou logiku od technologií a integračních detailů tak, aby se business pravidla nerozpádla při každé změně databáze, messaging vrstvy nebo externí služby. V `hiring_backend` se tento vzor promítá do kontraktů ve `src/shared/contracts/ports`, do technologických adaptérů ve `src/platform/*` i do rozdělení vstupních a výstupních hranic aplikace. Praktickým přínosem je, že doména vyjadřuje pouze schopnosti, které potřebuje, zatímco konkrétní infrastruktura zůstává uzavřena v adaptační vrstvě.

S hexagonálním uspořádáním úzce souvisí vzor `Repository`. Jeho smyslem je oddělit práci s perzistencí od aplikačních služeb tak, aby use-cases nemusely nést detailní znalost SQL dotazů a fyzického schématu. V backendu se tento přístup projevuje přímo v modulové struktuře `src/domain/*/repository`, kde každý kontext zapouzdřuje vlastní datový přístup. Přínosem je menší vazba mezi business logikou a databázovou vrstvou a současně čitelnější hranice odpovědnosti uvnitř jednotlivých modulů.

Dalším důležitým vzorem je `Dependency Injection`. Ten řeší problém skládání závislostí v aplikaci, která už není malým monolitickým skriptem, ale systémem s porty, adaptéry, službami a moduly s odlišnými rolmi. V `hiring_backend` tuto úlohu plní kontejner `Awilix` spolu s registrací vazeb v `src/container.registry.js`. Praktickým přínosem je, že jednotlivé části backendu nejsou pevně svázány konkrétní implementací při vytváření objektů, což zjednodušuje výměnu adaptérů, testování i dlouhodobou údržbu.

Pro zápisové a integrační toky je klíčový vzor `Transactional Outbox`. Řeší problém, jak bezpečně navázat vedlejší efekty na doménovou transakci bez rizika, že se business data uloží, ale navazující notifikace, audit nebo publikační událost se kvůli chybě nikdy nevykoná. V backendu se tento vzor realizuje pomocí tabulky `side_effect_outbox`, samostatného workeru a outbox handlerů. Praktickým přínosem je vyšší konzistence mezi databází a integrační vrstvou a menší náchylnost systému k chybám v mezistavech.

Na outbox navazuje vzor `Idempotency`. Ten řeší opačný, ale stejně důležitý problém: aby opakovaný požadavek nebo duplicitní zpracování nezpůsobilo opětovné provedení stejné zápisové operace. V textu backendu se tento přístup promítá zejména do `command_idempotency` a do řízení zápisových toků, kde je potřeba odlišit nový požadavek od opakovaného doručení. Praktickým přínosem je vyšší odolnost vůči nestabilní síti, opakovanému kliknutí uživatele nebo znovudoručení zprávy v integračním řetězci.

Posledním důležitým vzorem je `Adapter`, přesněji integrační mezivrstva blízka `Anti-Corruption Layer`. Řeší problém, jak ochránit doménu před cizími protokoly, datovými modely a chybovými stavy externích systémů. V `hiring_backend` se tento princip projevuje například přes `qualification-adapter` a `user-search-adapter`, zatímco doména pracuje pouze s kontrolovaným interním kontraktem. Praktickým přínosem je, že změna integračního detailu nebo specifik externího systému neprotéká přímo do business logiky backendu.

`hiring_backend` tedy nestojí na jednom izolovaném vzoru, ale na jejich kombinaci. Hexagonální uspořádání omezuje vazby mezi doménou a infrastrukturou, `Repository` abstrahuje přístup k datům, `Dependency Injection` řídí skládání závislostí, `Transactional Outbox` a `Idempotency` stabilizují integrační a zápisové toky a integrační adaptéry chrání doménu před cizími rozhraními. Právě tato kombinace umožňuje, aby backend zůstal čitelný i v situaci, kdy musí současně řešit business pravidla, bezpečnost, messaging a více externích vazeb.

5.2.2 IMPLEMENTACE DOMÉNOVÉ VRSTVY

Doménové jádro je členěno do modulů odpovídajících hlavním kontextům systému. Prakticky jde například o správu uchazečů (`applicants`), pracovních pozic (`jobs`), pohovorů (`interviews`), zaměstnanců (`employees`),

organizací (organizations), číselníků (catalog) nebo interních uživatelů (internal_users). Každý modul má vlastní use-cases, repozitáře a události, takže změna v jedné oblasti nemusí automaticky rozbít zbytek systému.

Toto členění řeší dva problémy současně. Zaprvé omezuje přímé závislosti mezi nesouvisejícími částmi backendu. Zadruhé umožňuje číst a rozvíjet konkrétní oblast bez nutnosti držet v hlavě celou aplikaci. Právě tato vlastnost je důležitá pro dlouhodobou udržitelnost projektu.

5.2.3 IMPLEMENTACE HEXAGONÁLNÍ ARCHITEKTURY

Architektonické principy rozebrané v předchozí kapitole se v implementaci promítají do konkrétní struktury kódu. `src/domain` nese doménové jádro, `src/shared/contracts/ports` explicitní port kontrakty a `src/platform` sekundární adaptéry; vazbu mezi nimi skládá Awilix v `composition` rootu a registru závislostí.

Vůči svému okolí doména komunikuje výhradně prostřednictvím explicitních kontraktů vytvářených pomocí helperu `createServicePort`, který zpřístupní jen povolené metody a uzamkne jejich rozhraní. Implementaci těchto odchozích závislostí následně realizují adaptéry jako `platform/qualification`, `platform/userSearch`, `platform/audit` a `platform/outbox`. Právě zde se doménové požadavky převádějí na interní HTTP volání, auditní transport nebo spolehlivé doručování vedlejších efektů, zatímco samotná doména zůstává odstíněna od toho, zda je fyzická vrstva realizována prostřednictvím SQL, HTTP, AMQP nebo objektového úložiště.

5.2.4 VYNUCENÍ ARCHITEKTONICKÝCH PRAVIDEL

Architektonická pravidla nestačí pouze deklarovat. Proto jsem je v implementaci částečně vynutil testy architektury. Ty ověřují, že doménové moduly neimportují repozitáře jiných modulů napřímo, že služby nepoužívají infrastrukturu mimo definované porty a že business vedlejší efekty nejsou volány mimo `outbox handlers`.

Tento mechanismus je důležitý hlavně proto, že architektonická kázeň má tendenci upadat právě v okamžiku, kdy projekt roste a přibývá tlak na rychlé úpravy. Testy zde fungují jako technická pojistka proti tomu, aby se výjimka z pravidla stala novým standardem.

5.2.5 IMPLEMENTACE SPOLEHLIVÉ ASYNCHRONNÍ KOMUNIKACE

Klíčovým implementačním problémem bylo zajistit, aby vedlejší efekty navázané na doménovou transakci byly spolehlivě doručeny i při chybách integrační vrstvy a současně nebyly prováděny duplicitně. Tento problém řeším kombinací vzorů Transactional Outbox a Idempotency, konkrétně tabulkou `side_effect_outbox`, samostatným workerem a řízením zápisových toků nad idempotency vrstvou. Doménová operace zapisuje business data i záznam do outboxu v jedné transakci až po commit fázi worker položku vyzvedne a publikuje.

Tabulka 5. 2: Typy outbox událostí implementovaných v backendu

event_type v outboxu	Implementační efekt
<code>email.welcome.v1</code>	Odeslání uvítacího e-mailu novému zaměstnanci
<code>email.raw.v1</code>	Odeslání generického e-mailu podle doménové události
<code>notification.role.v1</code>	Role-based fan-out notifikace v rámci organizace
<code>notification.user.v1</code>	Cílená notifikace konkrétnímu uživateli
<code>cv.publish.applicant.v1</code>	Publikace události pro AI analýzu CV uchazeče do RabbitMQ
<code>cv.publish.job_seeker.v1</code>	Publikace události pro AI analýzu CV zájemce
<code>job.embedding.requested.v1</code>	Publikace požadavku na AI zpracování job embeddingu

Worker používá dávkové zpracování, zamykání položek, řízené opakování a mrtvý stav pro neobnovitelné chyby. Přímé volání business vedlejších efektů je omezeno na outbox handlers; doménové služby pouze zapisují požadavek do outboxu. Tím se snižuje riziko nekonzistence mezi databází a integrační vrstvou.

Verzování událostí pomocí suffixu `v1`, případně dalších verzí, slouží k bezpečné evoluci integračního rozhraní. Pokud se změní struktura zprávy nebo její sémantika, lze zavést novou verzi bez nutnosti rozbít stávající konzumenty v jednom kroku.

5.2.6 IMPLEMENTACE AUDITNÍ VRSTVY

Auditní stopu nezapisuji synchronně v hlavní request cestě, ale přes samostatný worker `audit_writer_processor`. Ten konzumuje auditní události z RabbitMQ, validuje jejich payload a ukládá výsledek do tabulky `audit_events` v PostgreSQL. Tím zkracuji odezvu aplikačního API a současně snižuji riziko, že dočasné selhání transportní nebo databázové vrstvy zablokuje běžný transakční tok.

Z implementačního hlediska je důležité odlišovat vykonávací komponentu od datové struktury. `audit_writer_processor` představuje samostatnou zpracovatelskou službu navázanou na messaging vrstvu, zatímco `audit_events` je perzistentní tabulka, do níž se auditní stopa ukládá. Tato dvojice společně zajišťuje, že audit zůstává mimo kritickou request cestu, ale neztrácí vazbu na transakční dění systému.

5.3 IMPLEMENTACE VRSTVY INTELIGENTNÍHO ZPRACOVÁNÍ DAT

Vrstva inteligentního zpracování dat vznikla jako řešení na provozní tlak popsany v analytické části práce. V prostředí KZ probíhá nábor kontinuálně, takže systém souběžně pracuje s větším množstvím životopisů i průběžně upravovaných pracovních nabídek. Manuální screening životopisů je přitom časově náročný a snadno vede k přehlédnutí důležitých údajů. Transakční jádro proto nemá nést dokumentové a AI úlohy přímo.

Na tuto situaci odpovídám oddělenou vrstvou implementovanou jako samostatné služby v jazyce Go. Komunikace probíhá asynchronně přes RabbitMQ a inference zajišťuje Ollama s konfigurovatelným generativním modelem podle konkrétní služby a nasazení. Smyslem této vrstvy není řídit samotný náborový proces, ale doplnit jej o podpůrné schopnosti tam, kde by přímé zpracování dokumentů a textů zbytečně zatěžovalo backend.

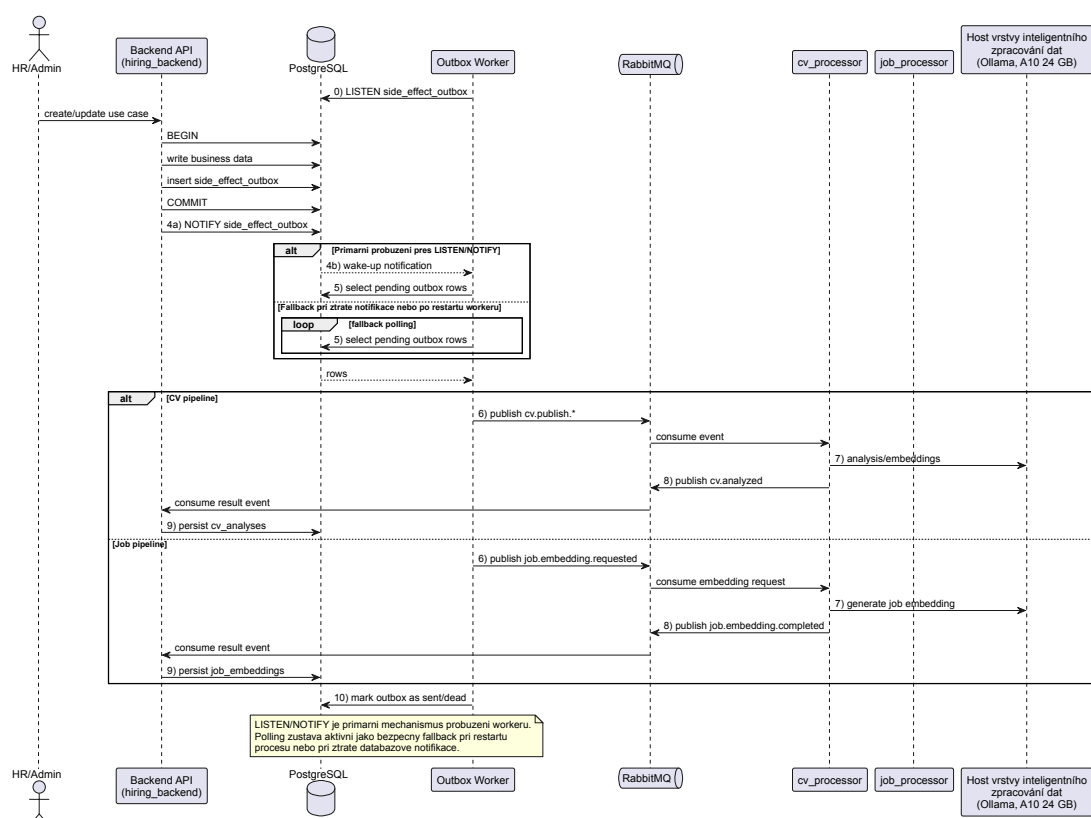
Služba `cv_processor` zpracovává příchozí životopisy a je navázána na asynchronní tok přes RabbitMQ. Po přijetí události stáhne dokument z objektového úložiště, extrahuje text přes Apache Tika, pomocí Ollama provede strukturované vytěžení a shrnutí a nad připraveným textem vytvoří embedding pomocí modelu `nomic-embed-text`. Výsledek pak vrací zpět do backendu přes RabbitMQ. Embeddingy zde neslouží jen jako doplňkové metadata, ale jako podklad pro sémantické porovnávání uchazečů a pozic. Backend je ukládá do `pgvector` jako `vector(768)`.

Služba `job_processor` řeší jiný typ úlohy. Vystupuje jako samostatná HTTP/SSE služba, kterou `hr-backend` volá synchronně při generování a úpravě textů pracovních nabídek. Také zde se používá Ollama, ale role služby je odlišná než

u `cv_processor`; zatímco jedna služba zpracovává dokumenty a vrací strukturované výstupy do asynchronního toku, druhá obsluhuje interaktivnější tvorbu textu blízkou uživatelské operaci.

Vedle toho běží samostatný asynchronní tok pro embeddingy pracovních pozic. Backend publikuje požadavek `job.embedding.requested.v1`, zpracování připraví text pozice do podoby vhodné pro vektorové porovnávání a výsledný embedding se vrací do tabulky `job_embeddings`. I zde se používá `nomic-embed-text`, takže sémantické porovnávání pracuje nad stejným typem reprezentace jako u životopisů. Oba procesory tak sdílejí stejný inferenční backend, ale plní odlišné implementační role.

Technické oddělení této vrstvy má přímý provozní důvod. Inference, práce s dokumenty a vektorové výpočty mají jiný latencní i chybový profil než transakční API. Jejich dočasná nedostupnost proto nesmí zablokovat základní náborové a onboardingové funkce. Systém tak degraduje řízeně. Uživatelé mohou přijít o část rozšířené funkcionality, nikoli o samotné transakční jádro.



Obrázek 5. 2: Realizační tok Outbox-RabbitMQ a vrstvy inteligentního zpracování dat v implementaci

5.4 IMPLEMENTACE DATOVÉ VRSTVY A MIGRACÍ

Perzistenci dat stavím na PostgreSQL 17 s rozšířením pgvector. Relační část pokrývá transakční agendu náboru a onboardingu, vektorová část podporuje scénáře inteligentního zpracování dat. Toto spojení řeší problém, jak držet konzistentně business data i sémantické reprezentace bez zavádění další samostatné databázové technologie.

V implementaci důsledně nesu organizační izolaci přes `organization_id` v klíčových entitách a kontroluji přístupový rozsah v aplikační vrstvě. Auditní stopu ukládám do samostatné struktury s omezením mutací, aby bylo možné doložit práci s citlivými daty i zpětně.

5.4.1 FYZICKÝ DATOVÝ MODEL

Na úrovni fyzického modelu pracuji s tabulkami odpovídajícími doménovým skupinám popsaným v architektuře. V náborové části jsou klíčové zejména `job_postings`, `job_roles`, `applicants`, `interview_events` a návazné pomocné tabulky pro obsah inzerátu, účastníky pohovorů a přílohy. Onboardingová část je realizována nad tabulkami `onboarding_workflows`, `onboarding_steps`, `user_onboarding_steps`, `onboarding_documents` a `user_documents`.

Přístupový model je fyzicky opřen o `users`, `user_roles`, `organization_memberships` a `resource_permissions`. Datová vrstva tak přímo nese organizační kontext i pravidla ReBAC, místo aby byla autorizace řešena jen dodatečně nad již načtenými daty.

Konkrétní datové typy volím podle charakteru uložených informací. Formuláře a odpovědi zaměstnanců ukládám jako `jsonb`, protože jejich struktura se může v čase měnit. Embeddingy životopisů a pracovních pozic ukládám jako `vector(768)` přes `pgvector`, aby bylo možné provádět sémantické porovnávání přímo v databázi.

5.4.2 MIGRACE SCHÉMATU

Schéma rozvíjím řízeně přes číslované SQL migrace. Tento přístup řeší problém, jak udržet databázový model auditovatelný a současně bezpečně nasaditelný do produkce. Běžné strukturální změny provádím transakčně standardními DDL příkazy, zatímco změny indexů pro živé prostředí odděluji do neblokujících kroků přes `CREATE INDEX CONCURRENTLY`.

Migrace zde zároveň fungují jako quality gate mezi novou binární verzí aplikace a stavem databáze. Backend tak nezačne běžet proti nekompatibilnímu

schématu, což je v systému s více službami a asynchronními vazbami zásadní bezpečnostní pojistka.

5.4.3 PERZISTENCE DOKUMENTŮ A INFRASTRUKTURNÍ TABULKY

Dokumenty uchazečů a zaměstnanců neukládám přímo do relační databáze. Tabulky jako `application_attachments`, `user_documents` nebo `onboarding_documents` obsahují metadata a reference na objektové klíče, zatímco fyzický obsah souborů je uložen v SeaweedFS přes S3 kompatibilní rozhraní. Tím řeším problém, jak zachovat výkonnou databázi pro transakční agendu a současně pracovat s většími soubory a jejich verzemi.

Vedle doménových tabulek používám i několik infrastrukturních tabulek. `side_effect_outbox` zajišťuje spolehlivé doručení vedlejších efektů po commit fázi, `command_idempotency` omezuje riziko duplicitního provedení zápisových operací a `audit_events` uchovává auditní stopu citlivých akcí. Tyto struktury nejsou business doménou samy o sobě, ale bez nich by nebylo možné naplnit požadavky na spolehlivost a auditovatelnost.

5.4.4 IMPLEMENTACE BEZPEČNOSTI A IDENTITY

Bezpečnostní implementaci stavím na více vrstvách. První vrstvou je autentizace uživatele přes session token, který middleware převádí na jednotný request kontext obsahující identitu, role a seznam organizací. Druhou vrstvou jsou endpoint guardy, které rozhodují, zda uživatel smí vstoupit do dané části API. Třetí vrstvou je datová autorizace v `hiring_backend`, realizovaná nad `resource_permissions` a návaznými vazbami na `organization_memberships`.

V praxi tím odděluji dvě různé otázky. Globální role uložená v `users.role_id` říká, jaký typ uživatele je přihlášen. Vztah ke konkrétní organizaci nebo pracovní pozici pak rozhoduje, k jakým datům skutečně smí přistoupit. `organization_memberships` proto neslouží jako zdroj role, ale jako evidence organizačních přístupů, jejich platnosti a způsobu přidělení.

Tabulka 5. 3: Globální role a jejich praktický význam v ReBAC modelu backendu

Role	Praktický vztah k datům
Běžný uživatel systému	Pracuje primárně se svými scénáři a nemá zvýšená administrativní oprávnění
Read-only role pro náborový dohled	Vidí jen organizace, kde má membership, a konkrétní pozice, ke kterým má přímé přiřazení
Operativní práce s nábořem	Má write přístup k organizaci a pracovním pozicím ve svěřeném závodě
Správa organizace a přístupů	Má admin přístup v rámci své organizace a může spravovat role i membershipy
Systémová provozní role	Má globální administrativní rozsah napříč organizacemi i provozními moduly

Role check v middleware tedy není poslední autorizační krok, ale jen vstupní filtr. Samotné čtení, změny a mazání zdrojů vyhodnocuji až na úrovni práce s daty nad `resource_permissions`. Dceřiné entity, například uchazeči, pohovory nebo přílohy, dědí oprávnění z nadřazené pracovní pozice nebo organizace, místo aby se pravidla duplikovala v každé tabulce zvlášť. Změny rolí a membershipů se navíc synchronizují asynchronně přes outbox události, aby model zůstal konzistentní i mimo kritickou request cestu.

5.5 IMPLEMENTACE ONBOARDINGOVÉHO PORTÁLU

Onboardingový portál jsem implementoval jako Next.js aplikaci oddělenou od veřejného kariérního portálu. Toto rozdělení řeší problém, že HR pracovníci a nastupující zaměstnanci potřebují pracovat s odlišnými typy úloh i s jiným bezpečnostním režimem. Na úrovni implementace jsem proto oddělil layouty, cesty i stavovou logiku podle role uživatele.

todo: rozšířit asi třeba o obrázek a nějaké kecy okolo

5.6 IMPLEMENTACE KARIÉRNÍHO PORTÁLU

Kariérní portál `kariera.kzcr.eu` jsem implementoval jako veřejný vstup do náborového procesu nad technologiemi Next.js, React a TypeScript. Frontend je oddělen od backendu záměrně. Veřejný portál řeší prezentaci obsahu, navi-

gaci a interakci s uchazečem, zatímco business pravidla pro práci s pozicemi, uchazeči a formulářovými daty zůstávají v jednom autoritativním API.

Domovská stránka propojuje obsahové a transakční scénáře. Uchazeč zde najde benefity, tematické kategorie pracovních rolí, mapový přehled nemocnic a vstup do katalogu volných míst. Samotný náborový tok tvoří seznam pozic, detail konkrétní nabídky a formulář reakce na vybranou pozici. Vedle toho portál obsahuje i samostatný kontakt pro zájemce bez vazby na konkrétní inzerát.

Komunikaci s backendem jsem soustředil do centralizované API vrstvy místo přímých HTTP volání z jednotlivých komponent. Tím řeším problém duplicitní síťové logiky v rozhraní a zjednodušuji změny v adresaci endpointů, timeoutu požadavků i mapování chybových stavů. Stav filtrů v katalogu pozic je navíc synchronizován s URL parametry, aby bylo možné konkrétní výběr sdílet a znovu otevřít ve stejném stavu.

Z pohledu uživatelské zkušenosti bylo důležité zachovat kontext při pohybu mezi seznamem a detailem pozice. Pro krátkodobý kontext používá portál `sessionStorage`, například pro návrat na stejné místo v seznamu, zatímco `localStorage` slouží k uchování vybraných pozic mezi jednotlivými navigacemi. Formulář reakce současně provádí klientskou validaci a používá idempotency klíč, aby se snížilo riziko duplicitních žádostí při opakovaném kliknutí nebo nestabilním spojení.

Vzhledem k tomu, že backend vrací popis pozice ve formátu HTML, řešil jsem i bezpečné vykreslení obsahu. HTML proto před zobrazením sanitizuji, aby se do stránky nedostal neověřený nebo škodlivý obsah. Jde o zdánlivý detail, ale právě na podobných místech se láme důvěryhodnost veřejného portálu.

5.6.1 INTEGRACE ANALYTICKÉHO NÁSTROJE UMAMI

Pro analytické účely jsem do portálu integroval nástroj Umami. Jeho smyslem není zasahovat do rozhodování systému, ale měřit průchod uživatelů portálem a identifikovat místa, kde uchazeči proces opouštějí. Inicializace analytiky je provedena centrálně v kořenové komponentě aplikace, aby bylo zajištěno jednotné měření napříč stránkami.

Sleduji zejména interakce s katalogem pozic, otevření detailu nabídky, zahájení reakce na pozici, práci s formulářem a výsledek jeho odeslání. Tato data slouží jako podpůrný vstup pro další iterace portálu a doplňují kvalitativní poznatky z pilotního uživatelského ověření.

5.7 IMPLEMENTACE DOHLEDOVÉ VRSTVY

Dohledovou vrstvu jsem implementoval jako kombinaci strukturovaného logování, metrik a centralizovaného dashboardingu. Tento krok řeší problém, že v distribuovanějším systému už nestačí číst lokální logy jednotlivých služeb. Potřebuji sledovat request kontext, zdraví asynchronní vrstvy i chování oddělených procesorů z jednoho místa.

Na úrovni aplikace generuji strukturované logy a publikuji metriky pro kritické toky, například autentizaci, outbox, messaging a vrstvu inteligentního zpracování dat. Tyto výstupy následně vstupují do samostatné monitorovací vrstvy popsané v architektonické kapitole. Dohled tedy nevzniká jako provozní doplněk přidáný na konci projektu, ale jako součást implementace hlavních toků.

5.8 KOMUNIKACE S NÁRODNÍMI REGISTRY

Napojení na národní registry jsem řešil primárně pro Národní registr zdravotnických pracovníků (NRZP) spravovaný ÚZIS. Praktická zkušenost ukázala, že samotné technické rozhraní ještě neznamena funkční integraci. Vedle klienta ského certifikátu bylo nutné řešit i přidělení externích identifikačních údajů a odpovídajících oprávnění na straně poskytovatele služby.

Z architektonického hlediska jsem proto integrační logiku neumístil přímo do `hiring_backend`, ale zarámoval ji jako adaptační mezivrstvu blízko vzoru `Adapter / Anti-Corruption Layer` a oddělil ji do interní služby `qualification-adapter`. Backend vystavuje pouze administrační endpoint `POST /api/v1/admin/qualifications/lookup`, který je dostupný vybraným rolím. Doménová služba podporuje dotaz podle čísla pracovníka v NRZP i podle rodného čísla, vstup normalizuje a validuje a teprve potom volá platformní adapter.

`qualification-adapter` běží pouze v interní Docker síti a funguje jako mezivrstva mezi backendem a existující integrační vrstvou nad InterSystems IRIS. Tento mezistupeň řeší tři problémy najednou: izoluje specifika SOAP/WSDL rozhraní mimo business logiku, sjednocuje chybové stavy do kontrolovaného HTTP kontraktu a zjednodušuje případnou výměnu integračního detailu bez zásahu do domény.

Výstup z registru backend převádí do jednotné doménové struktury obsahující identifikaci pracovníka a seznam odborných, specializovaných a zvláštních odborných způsobilostí. Současně vytváří auditní záznam o úspěšném i neúspěšném dotazu. Z důvodu ochrany osobních údajů se v auditu neukládá plné rodné číslo ani plné identifikátory dotazu, ale pouze jejich maskovaná podo-

ba a hash. Implementace tak spojuje automatizaci kontroly kvalifikace s provozní kontrolou nad citlivou integrační vazbou.

6 NAsAZENÍ SYSTÉMU DO CÍLOVÉHO PROSTŘEDÍ

Navržený systém má hodnotu jen tehdy, pokud jej lze provozně udržet v podmínkách KZ. Nasazení proto v této práci nevystupuje jako technický dovětek k implementaci, ale jako samostatná vrstva návrhu, která musí respektovat požadavky na on-premise provoz (NF07), dostupnost (NF04), auditovatelnost (NF11) a interoperabilitu (NF12). Nejde o uvedení jediné aplikace do běhu, ale o koordinaci spolupracujících frontendových, backendových, integračních a výpočetních služeb, které dohromady tvoří jeden provozní celek.

6.1 KONTEJNERIZAČNÍ MODEL A SÍŤOVÁ SEGMENTACE

Jednotlivé části systému distribuují jako samostatné kontejnerové image a provozují je pomocí Docker Compose. Tento přístup odpovídá cílovému on-premise prostředí a současně nevyžaduje zavedení rozsáhlejšího orchestračního rámce už v pilotní fázi. Bezstavové služby jsou aktualizovány výměnou image, zatímco stavové komponenty používají perzistentní volumes nebo sdílenou databázovou vrstvu. Přehled hlavních služeb uvádím v Tabulka 6. 1.

Tabulka 6. 1: Hlavní služby v nasazení

Služba	Role	Stav	Expozice
kariera.kzcrk	Kariérní portál pro publikaci inzerátů a podání přihlášek	Bezstavová	Veřejná
onboarding.kzcrk	Onboardingový portál pro HR a zaměstnance	Bezstavová	Interní web
hr-backend	Transakční API a integrační logika	Bezstavová (PostgreSQL, SeaweedFS)	Interní API
migration	Migrace databázového schématu	Bezstavová	Bez expozice
qualification-lookup-adapter	Lookup kvalifikací z NRZP	Bezstavová	Interní
user-search-adapter	Lookup interních uživatelů	Bezstavová	Interní
audit-writer	Zpracování událostí	Bezstavová (RabbitMQ → PostgreSQL)	Bez HTTP
PostgreSQL (pgvector)	Transakční a auditní data	Stavová	Interní
SeaweedFS	Úložiště dokumentů	Stavová	Interní
RabbitMQ	Fronta zpráv pro asynchronní zpracování	Stavová	Interní
Umami	Webová analytika	Stavová (PostgreSQL)	Omezená interní

Síťový model používá čtyři logické segmenty. app-network nese hlavní aplikační komunikaci mezi frontendy, backendem a stavovými službami. adapter-internal odděluje interní adaptéry qualification-adapter a user-search-adapter, které jsou přístupné pouze službě hr-backend. monitoring_network je vyhrazena pro sběr logů a metrik. Síť kz propojuje host vrstvy inteligentního zpracování dat s procesory cv_processor, job_processor, Apache Tika a Ollama. Interní adaptéry záměrně nemají publikovaný host port, protože jejich rozhraní slouží jen pro komunikaci mezi službami a nepatří do veřejného síťového perimetru.

Podrobnější síťová opatření, například konkrétní firewallová pravidla nebo umístění jednotlivých služeb do širší infrastruktury organizace, závisejí na cílovém prostředí KZ. V této práci proto zachycují především logické segmenty a důvody jejich oddělení, nikoli detailní síťovou konfiguraci konkrétní instalace.

6.2 NASAZENÍ APLIKAČNÍHO STACKU `hiring_backend`

Aplikační stack tvoří `hr-backend`, migrační služba `migration`, interní adaptéry `qualification-adapter` a `user-search-adapter`, stavové služby PostgreSQL, SeaweedFS, RabbitMQ a doprovodná analytická služba Umami. Toto rozdělení zachovává transakční API oddělené od integračních detailů i od pomocných provozních funkcí.

Startovací posloupnost má explicitní pořadí, protože právě na ní závisí, zda se nové nasazení rozběhne konzistentně vůči databázi a interním závislostem:

1. Nejprve se startuje PostgreSQL a čeká se na úspěšný healthcheck.
2. Následně se spustí jednorázová služba `migration`, která aplikuje SQL migrace a představuje quality gate mezi novou verzí aplikace a databázovým schématem.
3. Po úspěšné migraci se uvede do běhu interní adaptéry `qualification-adapter` a `user-search-adapter` a vyžadují jejich healthcheck.
4. Teprve poté se startuje `hr-backend`, navázané infrastrukturní služby a analytická služba Umami.
5. Při prvním nebo změněném nasazení Umami se doplňují jednorázové inicializační kroky `umami-db-init` a `umami-bootstrap`, které připraví databázi a základní konfiguraci analytického prostředí.

Význam migrační služby spočívá v tom, že brání startu API proti nekompatibilní verzi schématu. `qualification-adapter` a `user-search-adapter` běží jako interní Node.js služby s vlastním healthcheckem. Umami v tomto modelu nepředstavuje byznysové jádro systému, ale provozní doplněk pro měření používání portálů. Samotný `hr-backend` je zároveň připojen do `monitoring_network`, aby bylo možné centrálně sbírat jeho logy a metriky.

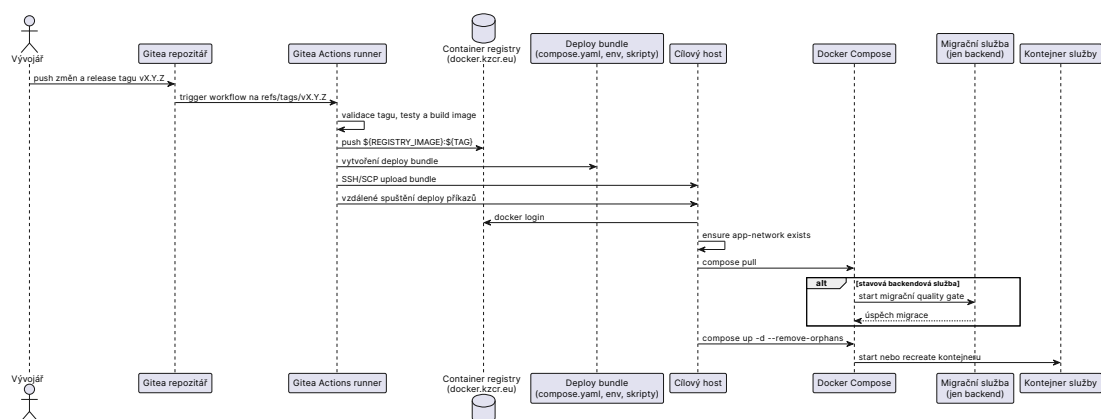
Z hlediska hexagonální architektury je důležité, že nasazovací hranice nejsou totožné s hranicemi portů a adaptérů. Většina vstupních adaptérů (`routes`, `kontrolery`, `middleware`) i většina výstupních adaptérů (`audit`, `outbox`, `storage`, `ReBAC`) běží uvnitř procesu `hr-backend` a nejsou samostatnými nasazovacími jednotkami. Samostatně nasazují pouze ty adaptéry a workery, které mají odlišný runtime, síťové oddělení nebo vlastní provozní životní cyklus, konkrétně `qualification-adapter`, `user-search-adapter`, `audit_writer_processor`, `cv_processor` a `job_processor`. `audit_writer_processor` je v tomto modelu provozován jako interní worker

bez veřejného HTTP rozhraní, zatímco vrstva inteligentního zpracování dat běží na samostatném výpočetním hostu.

6.3 RELEASE WORKFLOW, DISTRIBUCE IMAGE A AKTUALIZACE SLUŽEB

Nasazení celého ekosystému stavím na jednotném image-based CI/CD modelu. Vývojář po dokončení změny publikuje release tag `vX.Y.Z`, který aktivuje workflow na Gitea runneru. Ten ověří formát tagu, podle povahy služby provede build a testy, sestaví produkční image a publikuje jej do interního registru kontejnerových image `docker.kzcr.eu`.

Po úspěšném buildu workflow vytváří minimální deploy bundle. U frontendů jde typicky o `compose.yaml` a `.env.deploy`, u backendových a worker služeb se přidává runtime konfigurace nebo pomocný deploy skript. Bundle je přenesen přes SSH na cílový host, kde se následně provede `docker login`, `docker compose pull` a `docker compose up -d --remove-orphans`. Cílový server proto nepotřebuje pracovní checkout repozitáře; postačuje mu Docker, Docker Compose plugin, runtime konfigurace a přístup do interní registry.



Obrázek 6. 1: Obecný image-based release tok služby s distribucí do registry a nasazením na cílový host

U backendové části na tento obecný tok navazuje migrační quality gate. migration musí úspěšně dokončit změnu schématu ještě před startem hr-backend, jinak je nasazení ukončeno. U služeb vrstvy inteligentního zpracování dat zůstává princip shodný, ale deploy bundle navíc obsahuje runtime parametry pro přístup k RabbitMQ, SeaweedFS, Apache Tika a Ollama. Tyto služby současně běží na odděleném výpočetním hostu s vlastním runtime profilem a lokální GPU akcelerací, takže nejsou svázány se stejným provozním režimem jako transakční backend. Frontendy oproti tomu využívají hlavně build-time konfiguraci veřejných URL a analytiky.

Rollback je v tomto modelu realizován návratem ke staršímu release tagu a opětovným spuštěním Compose nad starším image. Výhodou image-based přístupu je, že rollback nevyžaduje nový build ze zdrojových kódů, ale pouze výběr dříve publikovaného artefaktu.

6.4 SPRÁVA KONFIGURACE A BEZPEČNOST PROVOZU

Konfiguraci služeb rozdělují do tří vrstev. První vrstvu tvoří build-time parametry frontendů, například veřejná URL API nebo parametry analytiky Umami, které vstupují přímo do sestavení výsledného image. Druhou vrstvu tvoří runtime konfigurace backendových, integračních a worker služeb, předávaná přes environment soubory na cílovém hostu. Třetí vrstvu představují server-only tajné údaje, které zůstávají mimo repozitář i mimo veřejně přenášené deploy bundle.

Tento model je důležitý z bezpečnostního hlediska. CI spravuje jen netajné parametry releasu a reference na image, zatímco citlivé hodnoty zůstávají odděleně na cílovém serveru. To se týká přístupových údajů k databázi, RabbitMQ, SMTP, objektovému úložišti i interním registrům. Interní služby současně používají oddělené neveřejné přístupové údaje pro vzájemnou komunikaci a integrační vazby. U vrstvy inteligentního zpracování dat obdobně oddělují netajnou runtime konfiguraci od tajných údajů pro RabbitMQ a SeaweedFS, aby release tok nezpřístupňoval citlivé provozní hodnoty.

6.5 DOHLEDOVÁ VRSTVA

Dohledovou vrstvu provozují odděleně od aplikační i výpočetní části vrstvy inteligentního zpracování dat. Tvoří ji Promtail, Loki, Prometheus a Grafana. Smyslem této vrstvy je soustředit provozní diagnostiku mimo business logiku a umožnit operátorům sledovat stav systému z jednoho místa.

Konkrétní role jednotlivých komponent dohledové vrstvy shrnuje Tabulka 6. 2; tato vrstva centralizuje logy, metriky a alerting mimo business logiku systému. Umami do ní nezařazují, protože jde o analytickou službu aplikace, nikoli o nástroj provozního dohledu.

Tabulka 6. 2: Role komponent dohledové vrstvy

Komponenta	Primární funkce	Provozní přínos
Promtail	Sběr a předzpracování logů z kontejnerů	Centralizovaná diagnostika bez zásahu do business logiky
Loki	Uložení a dotazování log streamů	Rychlá analýza incidentů podle času a štítků
Prometheus	Sběr metrik a trendové vyhodnocení	Podpora řízení dostupnosti a výkonu
Grafana	Vizualizace a upozornění	Jednotné operátorské rozhraní pro logy i metriky

6.6 PROVOZNÍ OMEZENÍ A MITIGACE

Navržený model nasazení představuje pragmatický kompromis mezi rychlostí implementace, provozní jednoduchostí a mírou infrastrukturní vyspělosti. Oproti jednodušší monolitické aplikaci přináší vyšší počet samostatných obrazů, více release artefaktů a nutnost koordinovat aplikační host s odděleným hostem vrstvy inteligentního zpracování dat. Provoz je navíc závislý na interním registru kontejnerových obrazů a na správné správě environment konfigurace na cílových serverech.

Další omezení plyne ze zvolené orchestrace pomocí Docker Compose. Tento přístup je dobře obhajitelný pro pilotní a menší produkční prostředí, ale neposkytuje vlastnosti běžné u vyšších orchestrátorů, například automatické rozložení zátěže mezi více uzly, nativní self-healing napříč hosty nebo deklarativní správu rozsáhlého clusteru. Část provozních rozhodnutí je navíc stále založena více na logové analýze než na plně formalizovaném modelu SLO, protože pokrytí všech komponent metrikami se průběžně rozšiřuje.

Tato omezení zmírňují kombinací několika opatření. Jednotný image-based CI/CD model s release tagy zjednodušuje build, distribuci i rollback. Healthchecky a migrační quality gate snižují riziko startu nefunkční verze. Oddělení tajných údajů od deploy bundle omezuje expozici citlivých hodnot. Samostatná dohledová vrstva a centralizace logů zase zlepšují diagnostiku incidentů v distribuovaném prostředí. Výsledný model proto považuji za přiměřený pro cílový on-premise provoz v podmínkách KZ.

7 UŽIVATELSKÉ TESTOVÁNÍ A ZPĚTNÁ VAZBA

Uživatelské ověření jsem zařadil do práce proto, že u personálního systému nestačí pouze technická správnost implementace. Pokud uživatel nerozumí stavu náboru, neví, kdo má provést další krok, nebo se v rozhraní ztrácí, procesní přínos systému se rychle vytrácí. Cílem této kapitoly proto není statisticky reprezentativní výzkum celé organizace, ale pilotní ověření toho, zda navržené řešení podporuje klíčové role v jejich každodenních scénářích a kde je ještě potřeba systém zpřesnit.

7.1 CÍL A RÁMEC PILOTNÍHO OVĚŘENÍ

Cílem pilotního uživatelského ověření bylo zodpovědět tři praktické otázky. První z nich se týkala průchodnosti hlavních scénářů bez nadměrné asistence. Druhá mířila na místa, kde se uživatelé zastavují, vracejí nebo si nejsou jisti významem dalšího kroku. Třetí sledovala, zda lze získanou zpětnou vazbu převést do konkrétních doporučení pro stabilizaci a další rozvoj řešení.

Na základě těchto cílů jsem formuloval následující výzkumné otázky:

1. Dokážou cílové role dokončit klíčové scénáře bez externí asistence?
2. Ve kterých krocích vzniká nejčastěji nejistota, zdržení nebo chybné rozhodnutí?
3. Které prvky rozhraní uživatelům pomáhají udržet kontext procesu a které jej naopak komplikují?
4. Jaká zjištění mají nejvyšší prioritu pro další iteraci systému?

7.2 METODICKÝ POSTUP A VÝBĚR RESPONDENTŮ

Pilotní uživatelské ověření jsem vedl jako moderované scénářové ověření. Každý účastník procházel scénáře odpovídající jeho roli, pracoval s daty připomínajícími reálný provoz a průběžně komentoval, jak rozumí jednotlivým krokům a stavům systému. Tento postup mi umožnil nesledovat pouze to, zda uživatel úkol dokončil, ale také proč se v určitém bodě zastavil a jak interpretoval význam zobrazených informací.

Z metodického hlediska jsem kombinoval tři vzájemně se doplňující techniky. Základem bylo scénářové ověření s jednoznačně definovaným cílovým stavem. Druhou vrstvou tvořilo průběžné komentování postupu uživatelem, které pomohlo odlišit pouhou chybu od hlubší nejasnosti v terminologii nebo navigaci. Třetí vrstvou byl krátký rozhovor po dokončení scénáře, ve kterém jsem ověřoval, co bylo pro uživatele přínosné, co považoval za problematické a jaké změny by očekával před širším provozním rozšířením.

Respondenty jsem vybíral účelově podle rolí, které reprezentují tři rozdílné pohledy na tentýž proces. Nešlo tedy o snahu pokrýt všechny varianty uživatelů v organizaci, ale ověřit, zda je systém srozumitelný z perspektivy operativní personální práce, manažerského dohledu a nástupu nového zaměstnance.

Tabulka 7. 1: Zastoupení rolí v pilotním uživatelském ověření

Role	Účel zapojení	Hlavní testovaná agenda
HR specialista	Operativní ověření náborového workflow	Správa uchazečů, změny stavů, plánování pohovorů, komunikace s kandidáty
Vedoucí pracovník	Ověření rozhodovacích a dohledových kroků	Hodnocení kandidátů, přehled stavu náboru, vstupní agenda nového zaměstnance
Nový zaměstnanec	Ověření srozumitelnosti onboardingového rozhraní	Plnění onboardingových kroků, práce s dokumenty, notifikace a orientace v úkolech

7.3 TESTOVACÍ SCÉNÁŘE A HODNOTICÍ KRITÉRIA

Testovací scénáře jsem navrhl tak, aby pokryly kritické průchody systémem od založení pracovní pozice až po onboarding nového zaměstnance. Každý scénář měl jasně definovaný cílový stav, protože právě ten rozhoduje o tom, zda uživatel úlohu skutečně dokončil, nebo pouze rozhraním prošel bez jistoty, co provedl.

Tabulka 7. 2: Sada testovacích scénářů

ID	Scénář	Primární role	Vazba na požadavky
S1	Založení a publikace nové pracovní pozice	HR	R2, R3
S2	Zpracování přihlášky a změna stavu uchazeče	HR	R3, R6
S3	Naplánování pohovoru a odeslání pozvánek	HR	R3
S4	Převod uchazeče na zaměstnance a spuštění onboardingu	HR	R4
S5	Vyplnění onboardingového kroku a nahrání požadovaných dokumentů	Zaměstnanec	R4
S6	Kontrola reportu náborové průchodnosti	Vedoucí	R6
S7	Ověření notifikace a reakce na přidělený úkol	HR / Zaměstnanec	R4, R6

Scénáře v Tabulka 7. 2 záměrně nesledují jen izolované klikací úlohy, ale uzly, ve kterých se rozhoduje o průchodnosti celého procesu. Pokud se uživatel ztratí při změně stavu uchazeče, při převodu na zaměstnance nebo při práci s onboardingovým úkolem, nevzniká pouze lokální nepohodlí v rozhraní, ale přímé zpomalení navazujících kroků.

Protože šlo o pilotní uživatelské ověření kvalitativního charakteru, nesoustředil jsem se na jedinou agregovanou metriku. Vhodnější bylo sledovat kombinaci ukazatelů, které společně vystihují, zda je proces pro uživatele čitelný, dokončitelný a provozně použitelný. Zajímalo mě zejména to, zda uživatel dosáhl cílového stavu, zda potřeboval zásah moderátora, kolik chybných kroků udělal a jak svůj postup následně slovně hodnotil.

Tabulka 7. 3: Ukazatele použité v pilotním uživatelském ověření

Metrika	Typ	Interpretace
Dokončení scénáře	Kvantitativní	Zda uživatel dosáhl cílového stavu bez zásahu do dat nebo návratu na začátek
Potřeba asistence	Kvantitativní	Počet situací, kdy bylo nutné vysvětlit význam stavu, kroku nebo navigace
Doba dokončení	Kvantitativní	Orientační čas potřebný k dosažení cílového stavu u jednotlivých scénářů
Chybové kroky a návraty	Kvantitativní	Počet chybných voleb, zbytečných návratů nebo slepých míst v rozhraní
Komentáře uživatelů	Kvalitativní	Slovní popis přínosů, nejistot a navrhovaných úprav
Závažnost zjištění	Expertní	Priorita opravy podle dopadu na průchod procesem a četnosti opakování

7.4 HLAVNÍ ZJIŠTĚNÍ A IMPLIKACE PRO DALŠÍ ROZVOJ

Získaná pozorování ukázala, že největší hodnotu uživatelé vnímali v centralizaci informací, ve sjednocení stavu kandidátů a v lepší dohledatelnosti odpovědností napříč náborovým a onboardingovým procesem.

Jako problematická se naopak ukázala místa, kde systém předpokládá vyšší procesní znalost uživatele, než jakou lze očekávat při prvním použití. Typicky šlo o potřebu jasněji vysvětlit význam stavů, lépe zvýraznit další očekávaný krok a oddělit informace, které jsou pouze informativní, od těch, které vyžadují akci. U onboardingových scénářů byla důležitá také viditelná vazba mezi úkolem, termínem a odpovědnou rolí. Bez ní se uživatel sice v systému orientuje, ale hůře chápe prioritu jednotlivých kroků.

Tabulka 7. 4: Šablona souhrnných výsledků pilotního uživatelského ověření

Scénář	Dokončení scénáře	Potřeba asistence	Medián času	Počet chyb	Priorita zjištění
S1	doplnit	doplnit	doplnit	doplnit	doplnit
S2	doplnit	doplnit	doplnit	doplnit	doplnit
S3	doplnit	doplnit	doplnit	doplnit	doplnit
S4	doplnit	doplnit	doplnit	doplnit	doplnit
S5	doplnit	doplnit	doplnit	doplnit	doplnit
S6	doplnit	doplnit	doplnit	doplnit	doplnit
S7	doplnit	doplnit	doplnit	doplnit	doplnit

Z hlediska dalšího rozvoje z toho plyne poměrně jednoznačný závěr. Nejbližší iterace nemají směřovat především k rozšiřování funkcí, ale k úpravám, které zpřesní terminologii, zvýrazní odpovědnost za další krok a lépe propojí úkol, termín a roli. Právě v těchto bodech se rozhoduje, zda systém skutečně zrychlí nábor a adaptaci, nebo zda pouze digitalizuje stávající nejistotu. Získaná zjištění proto v další kapitole přímo převádím do doporučení pro další směřování vývoje.

8 NÁVRH DALŠÍHO SMĚŘOVÁNÍ VÝVOJE

Tato kapitola navazuje na výsledky analýzy, architektonického návrhu, implementace, nasazení i pilotního uživatelského ověření systému. Jejím cílem není navrhnout co největší množství nových funkcí, ale určit takové pořadí kroků, které udrží procesní přínos řešení pro KZ, sníží provozní rizika a současně ponechá prostor pro další rozvoj bez zbytečného architektonického dluhu.

Navrhovaný rozvoj stavím na třech principech. Prvním je preference změn s přímým dopadem na každodenní práci HR pracovníků, vedoucích a nových zaměstnanců. Druhým je zachování architektonické kázně, aby rychlé provozní úpravy neoslabily modularitu systému. Třetím principem je měřitelnost. Každá další iterace má být vyhodnocena nejen podle toho, že byla nasazena, ale i podle toho, zda zlepšila průchodnost procesu, srozumitelnost rozhraní a provozní stabilitu.

8.1 PRIORITIZAČNÍ RÁMEC ROZVOJE

Pro potřeby plánování jsem další vývoj rozdělil do tří horizontů. Do krátkodobého, střednědobého a dlouhodobého. Toto členění mi umožňuje oddělit kroky, které je vhodné provést ihned po stabilizaci pilotního provozu, od změn, jež vyžadují širší integrační nebo organizační připravenost.

Prioritizace vychází z vazby na požadavky R1-R6 a NF01-NF12. Nejvyšší prioritou mají oblasti, které současně zvyšují provozní spolehlivost, bezpečnost a srozumitelnost práce se systémem. Pilotní uživatelské ověření navíc ukázalo, že další vývoj nemá směřovat pouze k novým funkcím, ale i k lepšímu vysvětlení stavů, odpovědností a návazností mezi kroky procesu.

8.2 KRÁTKODOBÁ FÁZE (0 - 6 MĚSÍCŮ)

Krátkodobý horizont by měl být zaměřen především na stabilizaci provozu a na odstranění nejednoznačností, které se ukázaly při pilotním ověření. První prioritou je proto sjednocení monitorování a provozního vyhodnocování napříč aplikační vrstvou a vrstvou inteligentního zpracování dat. Cílem není sbírat více technických dat samoúčelně, ale získat spolehlivý obraz o tom, kde vznikají latence, selhání nebo nedoručené asynchronní operace.

Druhou prioritou je zpřesnění uživatelského rozhraní v bodech, kde testování ukázalo zvýšenou míru nejistoty. Prakticky to znamená lépe vysvětlit význam stavů kandidáta, viditelněji zobrazit odpovědnost za další krok vstupní agendy a seskupit onboardingové úkoly podle procesu, nikoli pouze podle technického typu záznamu. Tyto úpravy nejsou architektonicky rozsáhlé, ale mají okamžitý dopad na reálné používání systému.

Třetí oblastí krátkodobého rozvoje je bezpečnostní a provozní hygiena prostředí. Patří sem pravidelná rotace přístupových údajů, zpřesnění práce s tajnými hodnotami, kontrola obnovitelnosti záloh a formalizace postupů při nedostupnosti vrstvy inteligentního zpracování dat. Jde o změny, které nejsou z pohledu uživatele nejviditelnější, ale mají zásadní význam pro důvěryhodnost a dlouhodobou provozuschopnost systému.

8.3 STŘEDNĚDOBÁ FÁZE (6 - 18 MĚSÍCŮ)

Ve střednědobém horizontu má smysl soustředit se na rozšíření funkcí, které zvyšují řídicí hodnotu systému. První oblastí je rozvoj reportingu a procesní analytiky. Vedení organizace potřebuje sledovat nejen počet otevřených pozic, ale i průchodnost náboru, dobu reakce mezi kroky, úspěšnost adaptačních plánů a rozdíly mezi odštěpnými závody.

Druhou oblast představuje další rozvoj integračních vazeb systému, zejména ve vztahu k externím registrům a navazujícím interním agendám (Vema, Plánování služeb, atd.). Rozvoj by měl zůstat řízený přes explicitní integrační kontrakty, aby nedocházelo k tomu, že externí systémy začnou určovat podobu doménového modelu.

Třetím směrem střednědobého rozvoje je vyšší automatizace vstupní agendy a adaptace. Přínos zde nepřinese jen více notifikací, ale hlavně lepší práce s pravidly, eskalacemi a přehledem nesplněných kroků. Cílem je omezit situace, kdy je úkol sice v systému založen, ale z pohledu uživatele není zřejmé, kdo jej má převzít a co blokuje další postup.

8.4 DLOUHODOBÁ FÁZE (18+ MĚSÍCŮ)

Dlouhodobý rozvoj by měl být veden jako řízená evoluce již zavedené hybridní architektury. Nejde tedy o přechod k hybridnímu modelu, protože ten je součástí řešení už nyní, ale o rozhodování, zda mají být některé další části systému postupně oddělovány podle skutečných provozních dat. Takový krok dává smysl pouze tehdy, pokud konkrétní komponenta dlouhodobě vykazuje odlišný rytmus změn, zátěžový profil nebo provozní požadavky než zbytek jádra.

Vedle architektonické evoluce bude v delším horizontu důležitější i datové governance. S růstem objemu historických dat poroste potřeba jednotně definovat metriky, popsat význam reportovaných ukazatelů a určit odpovědnost za jejich interpretaci. Bez této vrstvy by se i technicky kvalitní systém postupně měnil v další zdroj nejednoznačných dat.

Samostatnou dlouhodobou oblastí je také kapacitní a kvalitativní rozvoj vrstvy inteligentního zpracování dat. Pokud se její výstupy stanou běžnou součástí náboru a adaptace, bude nutné systematicky vyhodnocovat přesnost, latenci, provozní náklady i dopady případných chyb. Rozvoj této vrstvy proto nesmí být veden jen technologickou atraktivitou, ale jasně měřeným přínosem pro proces.

8.5 ŘÍZENÍ ZMĚNY A ORGANIZAČNÍ PŘEDPOKLADY

Úspěch dalšího rozvoje nebude záviset jen na technických rozhodnutích, ale i na tom, zda organizace udrží pravidelný cyklus vyhodnocování změn. Do tohoto cyklu musí vstupovat vývoj, provoz i vlastníci HR procesů. Bez společného vyhodnocení hrozí, že technické priority začnou sledovat interní pohodlí týmu místo reálných problémů uživatelů.

Stejně důležitá je průběžná práce se zpětnou vazbou. Pilotní uživatelské ověření ukázalo, že i drobná nejasnost v názvu stavu nebo ve zobrazení odpovědnosti může zpomalit celý proces více než absence nové funkce. Změny proto doporučuji nasazovat inkrementálně, ověřovat je na konkrétních scénářích a teprve poté rozšiřovat do širšího provozu.

8.6 ZÁVĚREČNÉ DOPORUČENÍ KAPITOLY

Pro další směřování vývoje považuji za nejvhodnější evoluční postup. Nejprve je potřeba stabilizovat provoz a odstranit bariéry zjištěné při uživatelském ověření, poté rozšiřovat analytiku, integrace a automatizaci a teprve nakonec otevírat otázku hlubších strukturálních změn. Takto zvolený postup dává nejvyšší šanci, že systém bude dlouhodobě podporovat cíle digitalizace náboru a adaptace bez zbytečného provozního rizika.

Navržený plán současně zachovává prostor pro budoucí organizační i technické změny v KZ, aniž by narušil základní architektonické principy stanovené v této práci.

9 ZÁVĚR

Tato diplomová práce vycházela z praktického problému, který se v prostředí Krajské zdravotní, a.s. dlouhodobě projevoval jako roztržitá správa náboru a adaptace pracovníků. Analytická část ukázala, že hlavním omezením není jediný nefunkční krok, ale souběh několika faktorů, mezi které patří rozptýlená data, ruční komunikace mezi rolemi, nízká transparentnost stavu náboru, chybějící auditní stopa a papírově vedená adaptace nových zaměstnanců. V organizaci s více odštěpnými závody a vysokou frekvencí nástupů se tyto slabiny navzájem zesilují a postupně snižují propustnost celého procesu.

Na základě zjištěného stavu jsem formuloval požadavky na cílové řešení a porovnal je s dostupnými komerčními produkty. Rešerše ukázala, že na trhu existují kvalitní dílčí nástroje pro správu náboru nebo onboarding, ale žádné z hodnocených řešení současně nenaplnuje požadavek na procesní kontinuitu, provoz na vlastní infrastrukturu, multi-tenantní členění organizace a vazbu na specifika českého zdravotnického prostředí. Tato zjištění vedla k rozhodnutí navrhnout a dále rozvíjet vlastní řešení, které dokáže uvedené požadavky spojit v jednom datovém a procesním modelu.

V návrhové části byla představena hybridní architektura kombinující modulární monolit pro transakční jádro a oddělenou vrstvu pro inteligentní zpracování dat. Tento přístup umožňuje zachovat provozní jednoduchost systému a zároveň podporuje jeho rozšiřitelnost, auditovatelnost a integrační schopnosti. Součástí práce je také implementace backendu, kariérního portálu, onboardingového rozhraní, integračních adaptérů a nasazení v on-premise prostředí.

Uživatelské ověření potvrdilo, že hlavním přínosem řešení je sjednocení informací, zvýšení transparentnosti procesu a omezení ruční komunikace mezi zúčastněnými rolemi. Zároveň se ukázalo, že pro úspěch systému je klíčová nejen jeho technická kvalita, ale i srozumitelnost procesů, jasné rozdělení odpovědností a průběžná práce se zpětnou vazbou uživatelů.

Hlavním přínosem práce je propojení analytického, architektonického a implementačního pohledu na konkrétní problém organizace. Výsledkem není pouze návrh, ale prakticky využitelný základ platformy respektující specifika zdravotnického prostředí. Limitem práce zůstává nutnost dalšího rozvoje systému v návaznosti na provozní zkušenosti a dostupné kapacity.

Práce nicméně prokazuje, že vlastní softwarové řešení představuje v daném kontextu efektivnější alternativu k rigidním komerčním systémům. Zároveň poskytuje metodický rámec pro realizaci obdobných digitálních transformací ve zdravotnických organizacích.

POUŽITÁ LITERATURA

- [1] ŘEPA, Václav. *Podnikové procesy: procesní řízení a modelování*. 2., aktualiz. a rozš. vyd. Praha: Grada, 2007. ISBN 978-80-247-2252-8. 281 s.
- [2] MINISTERSTVO ZDRAVOTNICTVÍ ČR. *Jak zajistit dostatek zdravotnického personálu? Česká republika a WHO hledají řešení pro budoucnost*. Online. 7. 2025. Dostupné z: <https://mzd.gov.cz/tiskove-centrum-mz/jak-zajistit-dostatek-zdravotnickeho-personalu-ceska-republika-a-who-hledaji-reseni-pro-budoucnost/>. [citováno 2026-02-08].
- [3] TEAMIO. *Automatic import of vacancies*. Online. 2026. Dostupné z: <https://www.teamio.com/en/what-it-can-do/automatic-import-of-vacancies/>. [citováno 2026-02-27].
- [4] TEAMIO. *Pricing*. Online. 2026. Dostupné z: <https://sk.teamio.com/en/pricing/>. [citováno 2026-02-27].
- [5] TEAMIO. *Vyskusat zadarmo*. Online. 2026. Dostupné z: <https://sk.teamio.com/vyskusat-zadarmo/>. [citováno 2026-02-27].
- [6] RECRUITIS.IO. *ATS Recruitis: Naborovy software*. Online. 2026. Dostupné z: <https://www.recruitis.io/>. [citováno 2026-02-27].
- [7] DATACRUIT. *Modern recruitment software with AI*. Online. 2026. Dostupné z: <https://www.datacruit.com/>. [citováno 2026-02-27].
- [8] DATACRUIT. *Price list of the Datacruit ATS system*. Online. 2026. Dostupné z: <https://www.datacruit.com/en-gb/pricing>. [citováno 2026-02-27].
- [9] SLONEEK. *Recruitment Platform (ATS) - Candidate Management*. Online. 2026. Dostupné z: <https://www.sloneek.com/ats/>. [citováno 2026-02-27].
- [10] ONBEE.APP. *Onboarding and Offboarding; digitalni karta zamestnan-ce, HRIS*. Online. 2026. Dostupné z: <https://onbee.app/cz/>. [citováno 2026-02-27].
- [11] SAP. *What is SAP SuccessFactors?*. Online. 2026. Dostupné z: <https://www.sap.com/products/hcm/what-is-sap-successfactors.html>. [citováno 2026-02-27].
- [12] SAP. *SAP SuccessFactors Onboarding*. Online. 2026. Dostupné z: <https://www.sap.com/products/hcm/employee-onboarding.html>. [citováno 2026-02-27].

- [13] WORKDAY. *Unified Solutions*. Online. 2026. Dostupné z: <https://www.workday.com/en-us/products/human-capital-management/unified-solutions.html>. [citováno 2026-02-27].
- [14] WORKDAY. *Talent Acquisition and Recruiting Software*. Online. 2026. Dostupné z: <https://www.workday.com/en-us/products/talent-management/talent-acquisition.html>. [citováno 2026-02-27].
- [15] ORACLE. *Human Capital Management (HCM)*. Online. 2026. Dostupné z: <https://www.oracle.com/human-capital-management/>. [citováno 2026-02-27].
- [16] ORACLE. *Oracle Recruiting and Recruiting Booster*. Online. 2026. Dostupné z: <https://www.oracle.com/human-capital-management/recruiting/>. [citováno 2026-02-27].
- [17] PORTAL GOV.CZ. *Overeni udaju o zdravotnickem pracovníkovi*. Online. 2026. Dostupné z: <https://portal.gov.cz/sluzby-vs/vypis-udaju-o-zdravotnickem-pracovnikovi-S1348>. [citováno 2026-02-27].
- [18] BASS, Len; Paul CLEMENTS a Rick KAZMAN. *Software Architecture in Practice*. 2. Addison-Wesley, 2003. ISBN 978-0-321-15495-8.
- [19] NEWMAN, Sam. *Building Microservices: Designing Fine-Grained Systems*. 2. O'Reilly Media, 2019. ISBN 978-1-492-03402-5.
- [20] COCKBURN, Alistair. *Hexagonal architecture*. Online. 2005. Dostupné z: <https://alistair.cockburn.us/hexagonal-architecture/>. [citováno 2026-03-17].
- [21] EVANS, Eric. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003. ISBN 978-0-321-12521-7.
- [22] EVANS, Eric. *DDD Reference: Definitions and Pattern Summaries*. Online. 2015. Dostupné z: https://www.domainlanguage.com/wp-content/uploads/2016/05/DDD_Reference_2015-03.pdf. [citováno 2026-04-07].